Internet of Things: Protocols and Networks (CSC2106)

CONTRAINED APPLICATION PROTOCOL

CoAP
RFC 7252

MQTT

Comparing Application Protocol …

	DDS	AMQP	CoAP	MQTT	REST / HTTP	XMPP
TRANSPORT	UDP/IP (unicast + multicast) TCP/IP	TCP/IP	UDP/IP	TCP/IP	TCP/IP	TCP/IP
INTERACTION MODEL	Publish-and-Subscribe, Request-Reply	Point-to-Point Message Exchange	Request-Reply (REST)	Publish-and-Subscribe	Request-Reply	Point-to-Point Message Exchange
SCOPE	Device-to-Device Device-to-Cloud Cloud-to-Cloud	Device-to-Device Device-to-Cloud Cloud-to-Cloud	Device-to-Device	Device-to-Cloud Cloud-to-Cloud	Device-to-Cloud Cloud-to-Cloud	Device-to-Cloud Cloud-to-Cloud
AUTOMATIC DISCOVERY	✓	-	✓	-	-	-
CONTENT AWARENESS	Content-based Routing Queries	-	-	-	-	-
QoS	Extensive (20+)	Limited	Limited	Limited	-	-
INTEROPERABILITY LEVEL	Semantic	Structural	Semantic	Foundational	Semantic	Structural
SECURITY	TLS, DTLS, DDS Security	TLS + SASL	DTLS	TLS	HTTPS	TLS + SASL
DATA PRIORITIZATION	Transport Priorities	-	-	-	-	-
FAULT TOLERANCE	Decentralized	Implementation-Specific	Decentralized	Broker is SPoF	Server is SPoF	Server is SPoF

https://devopedia.org/constrained-application-protocol

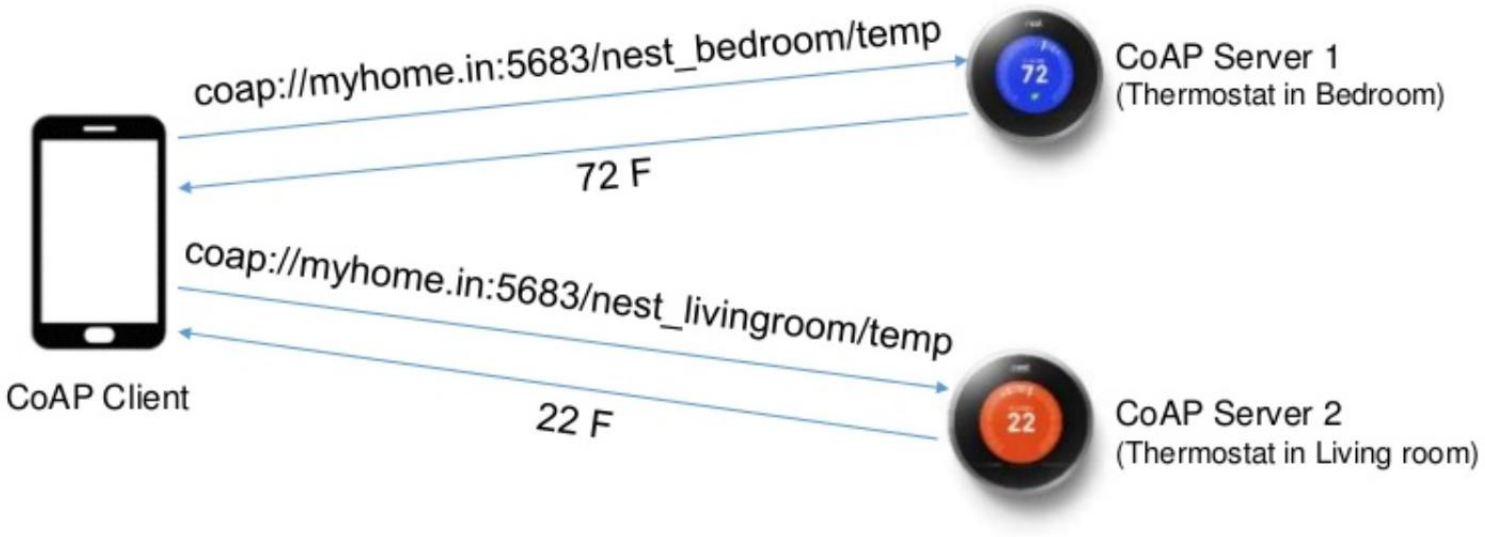
CoAP

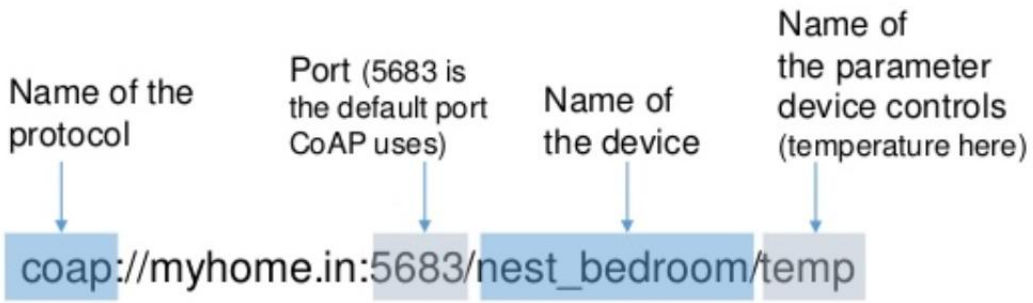
RFC 7252

Constrained Application Protocol (CoAP)

- Developed by the Constrained Resource Environments (CoRE) IETF group
- REST-based web transfer protocol (RESTful protocol)
- To communicate over the Internet using UDP instead of TCP (while still providing reliability)
- CoAP has been designed to work on resource-constrained microcontrollers with as low as 10 KB of RAM and 100 KB of code space (RFC 7228).
- CoAP could be used for adapting simple HTTP interfaces into a more compact protocol
  - different message types to request resources from server: GET, PUT, POST, DELETE and **OBSERVE**
  - subset of HTTP functionality re-designed for low power embedded devices such as sensors (for IoT and M2M)
- Supports multicast and congestion control

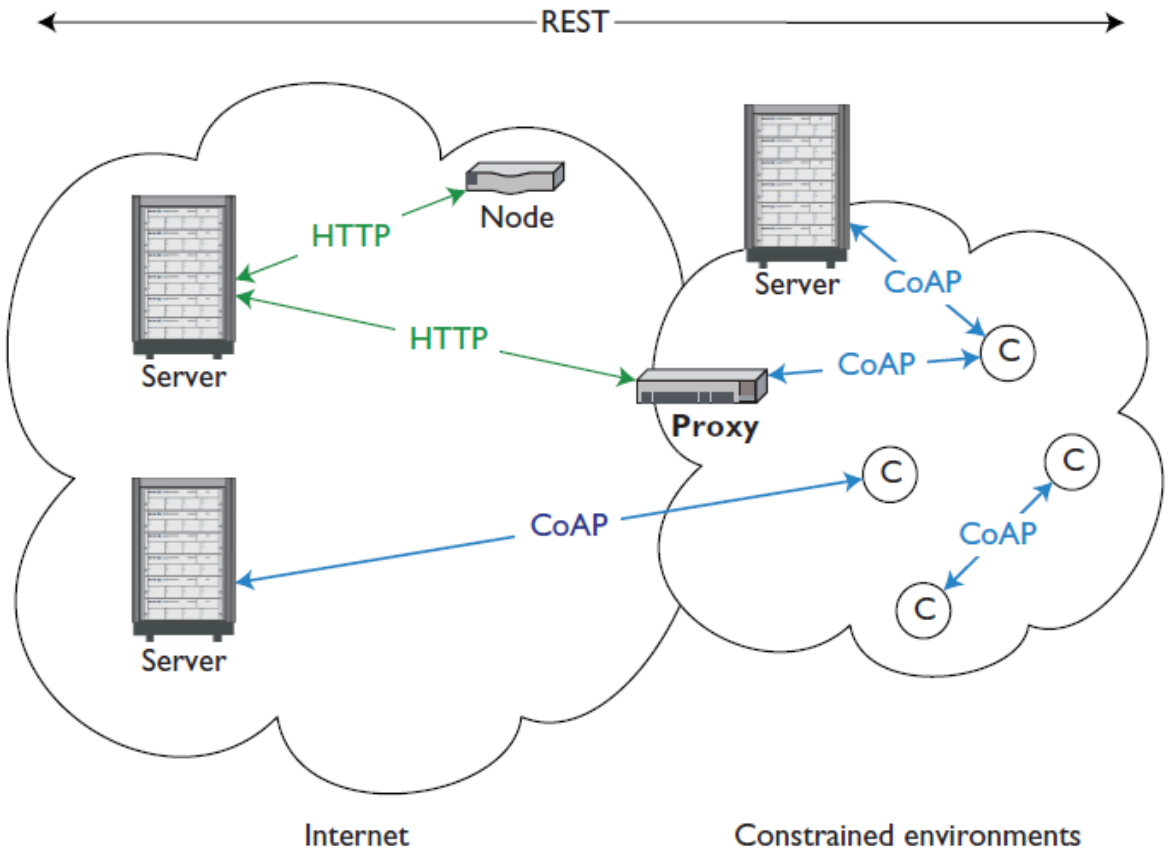
Example of a CoAP Transaction



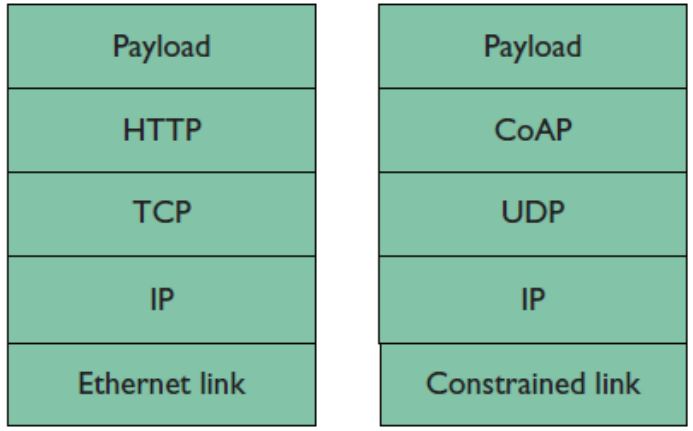


https://devopedia.org/constrained-application-protocol

CoAP Architecture



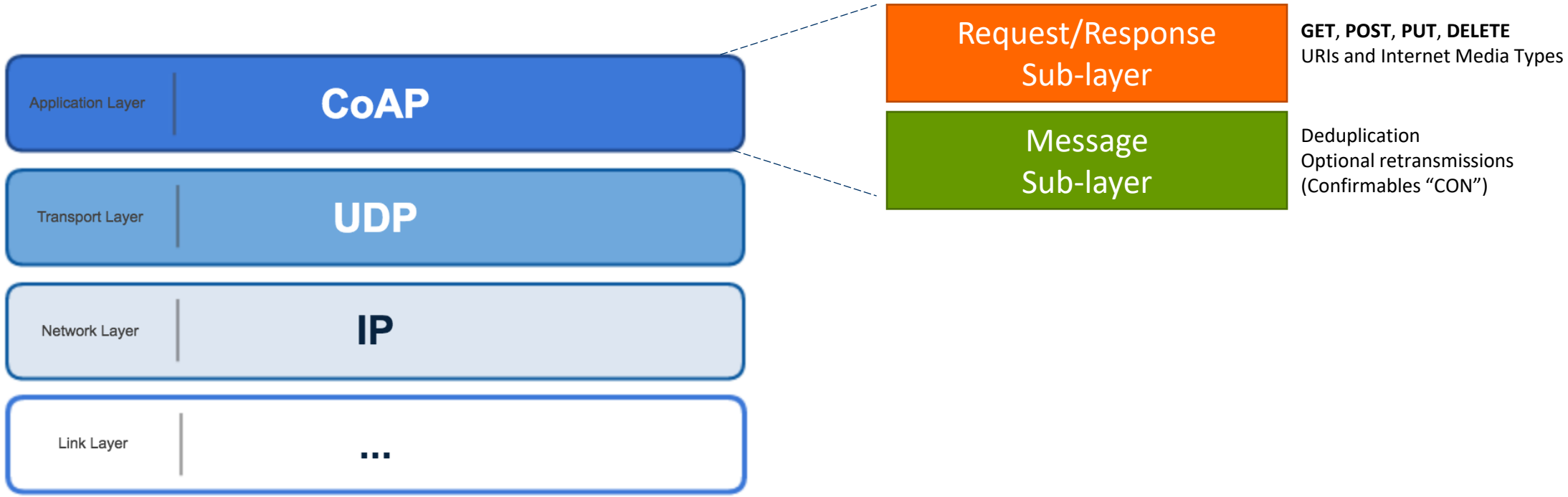
HTTP and CoAP work together across constrained and traditional Internet environments



HTTP protocol stack and CoAP protocol stack

Bormann, Carsten, Angelo P. Castellani, and Zach Shelby. 2012. "CoAP: An application protocol for billions of tiny Internet nodes." IEEE Internet Computing, vol. 1, no. 2, pp. 62–67, Mar/Apr.

CoAP Stack


Application Layer
CoAP
Transport Layer
UDP
Network Layer
IP
Link Layer
...
Request/Response Sub-layer
GET, POST, PUT, DELETE
URIs and Internet Media Types
Message Sub-layer
Deduplication
Optional retransmissions
(Confirmables "CON")

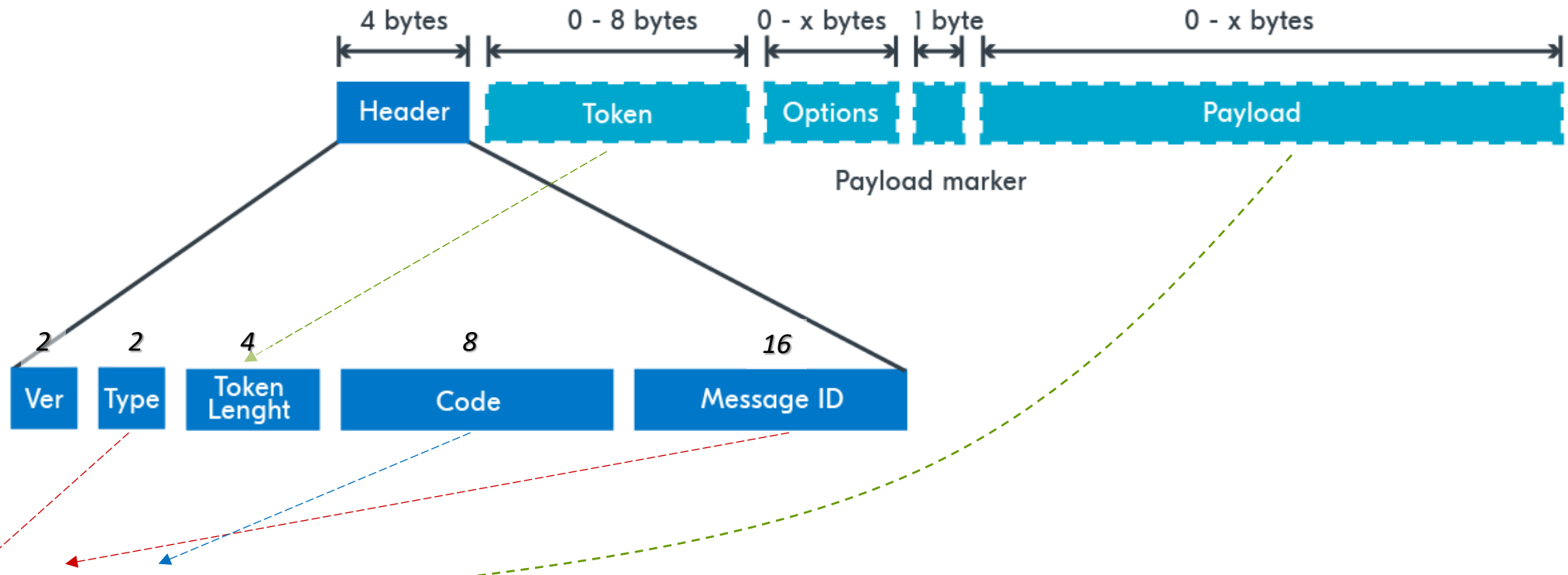
Message Header (4 bytes)

Bit:	0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7
	Ver, T, TKL	Code	Message ID	
	Token (if any)			
	Options (if any)			
	Payload Marker	Payload (if any)		

Ver - Version (1)
T – Message Type (Confirmable, Non-Confirmable, Acknowledgement, Reset)
TKL- Token Length, if any, the number of Token bytes after this header
Code - Request Method (1-10) or Response Code (40-255)
Message ID – 16-bit identifier for matching responses
Token – Optional response matching token

https://tools.ietf.org/html/rfc7252
https://tools.ietf.org/id/draft-ietf-core-coap-00.html

Message Mapping



CON [0x1A] GET /humidity

ACK [0x1A] 2.05 Content “<humidity>…”

Response Code

CoAP Status Code	Description
2.01	Created
2.02	Deleted
2.03	Valid
2.04	Changed
2.05	Content
2.31	Continue
4.00	Bad Request
4.01	Unauthorized
4.02	Bad Option
4.03	Forbidden
4.04	Not Found
4.05	Method Not Allowed
4.06	Not Acceptable
4.08	Request Entity Incomplete
4.12	Precondition Failed
4.13	Request Entity Too Large
4.15	Unsupported Content-Format
5.00	Internal Server Error
5.01	Not Implemented
5.02	Bad Gateway
5.03	Service Unavailable
5.04	Gateway Timeout
5.05	Proxying Not Supported

HTTP Status Code	Description
1xx	Informational
2xx	Successful 200 – OK 201 – Created 202 – Accepted 204 – No Content
3xx	Redirection 301 - Moved Permanently 305 - Use Proxy 307 - Temporary Redirect
4xx	Client Error 400 – Bad Request 401 – Unauthorized 403 – Forbidden 404 - Not Found 405 – Method Not Found 408 – Request Timeout
5xx	500 – Internal Server Error 501 – Not Implemented 503 – Service Unavailable 504 - Gateway Timeout

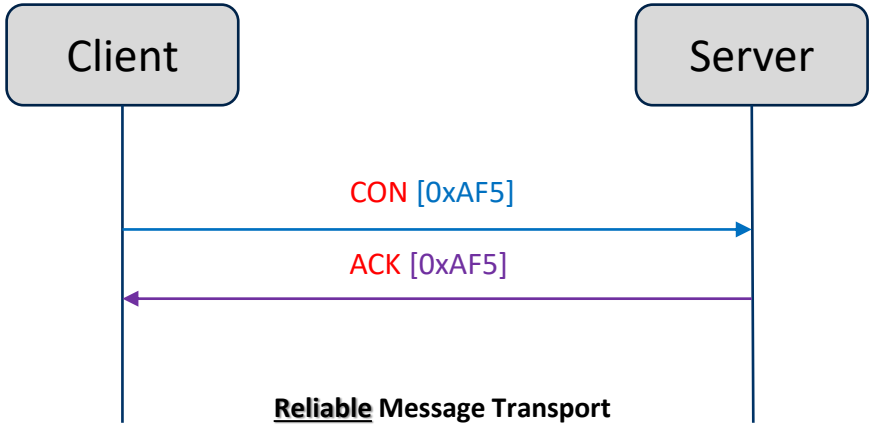
Only mostly used HTTP Status Codes are listed here

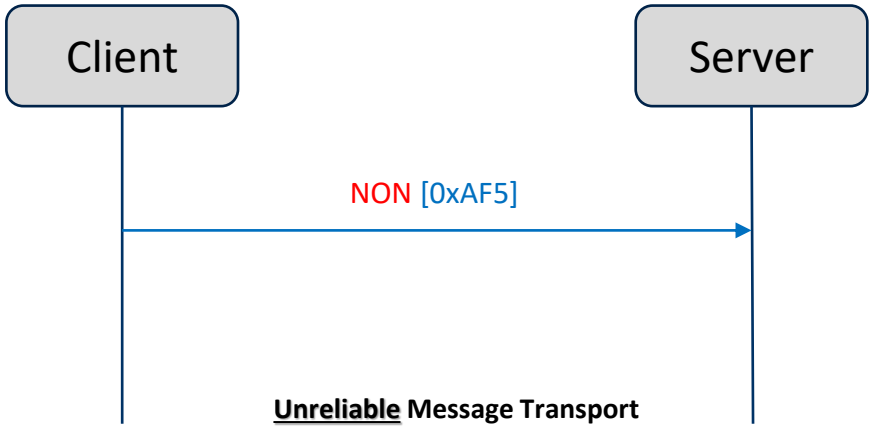
https://www.slideshare.net/HamdamboyUrunov/the-constrained-application-protocol-coap-part-2

Message Layer Model

Four Message Types:

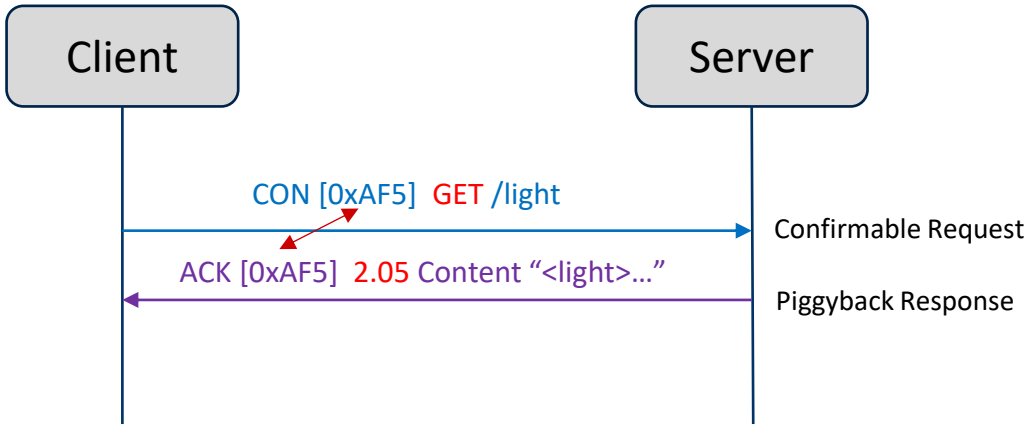
- **Confirmable (CON)** – requires an acknowledgement
- **Non-confirmable (NON)** – no acknowledgement needed
- **Acknowledgement (ACK)** – Acknowledge a Confirmable message
- **Reset (RST)** - indicates a Confirmable message has been received but context is missing for processing



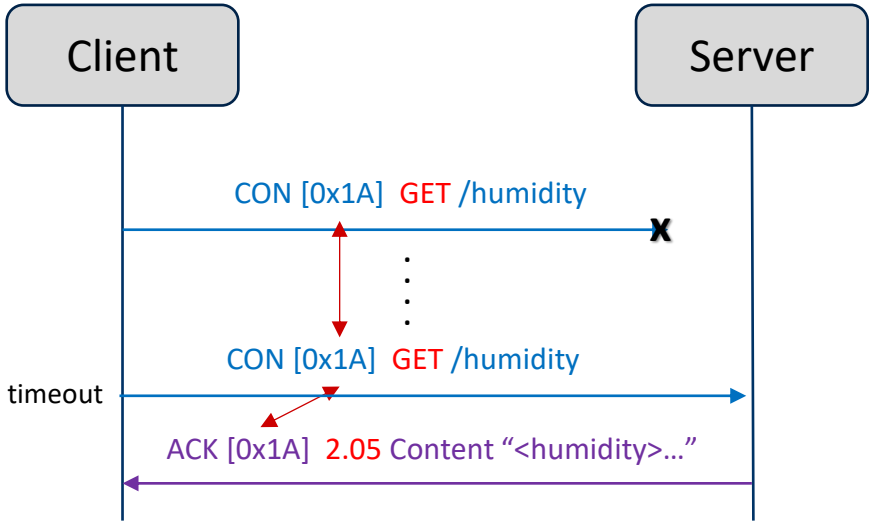


Request / Response Layer Model

Piggy-backed

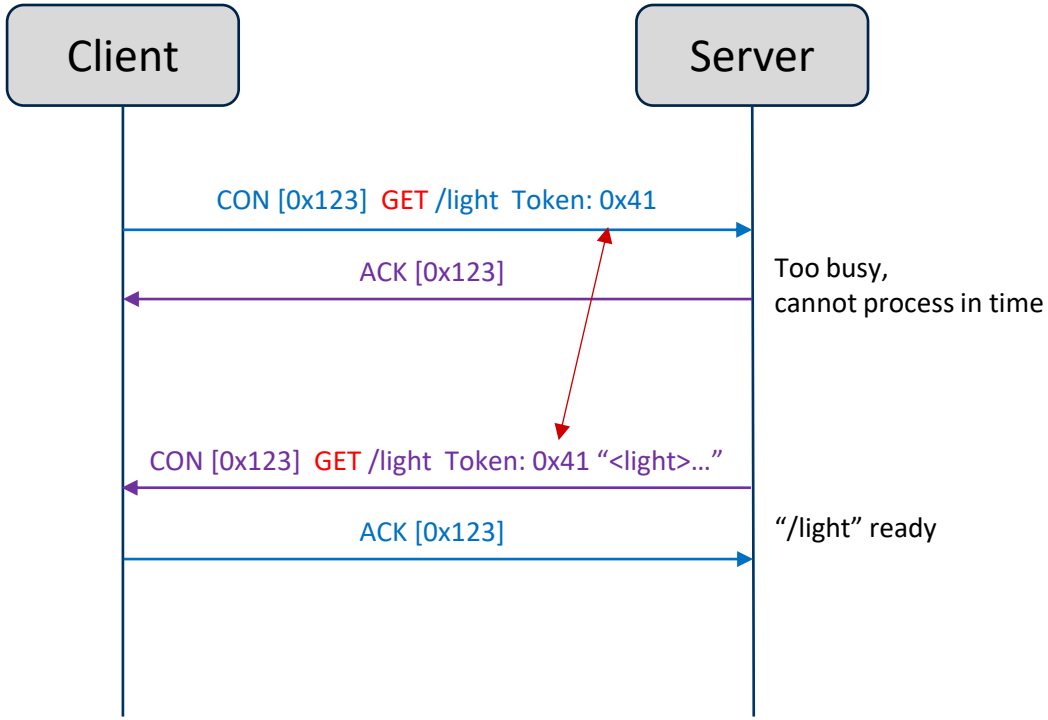


Dealing with Packet Loss

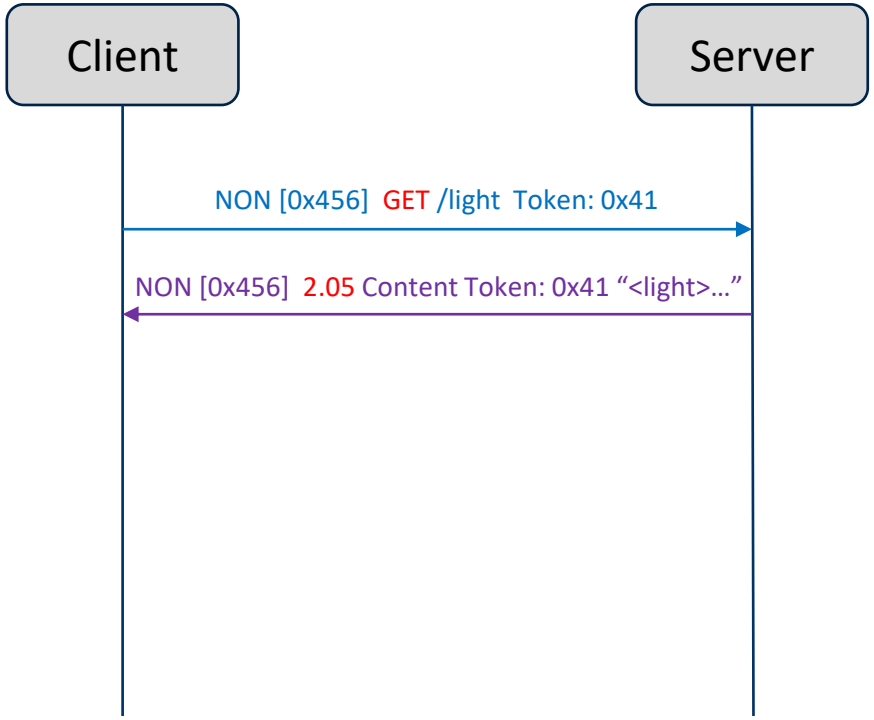


Request / Response Layer Model

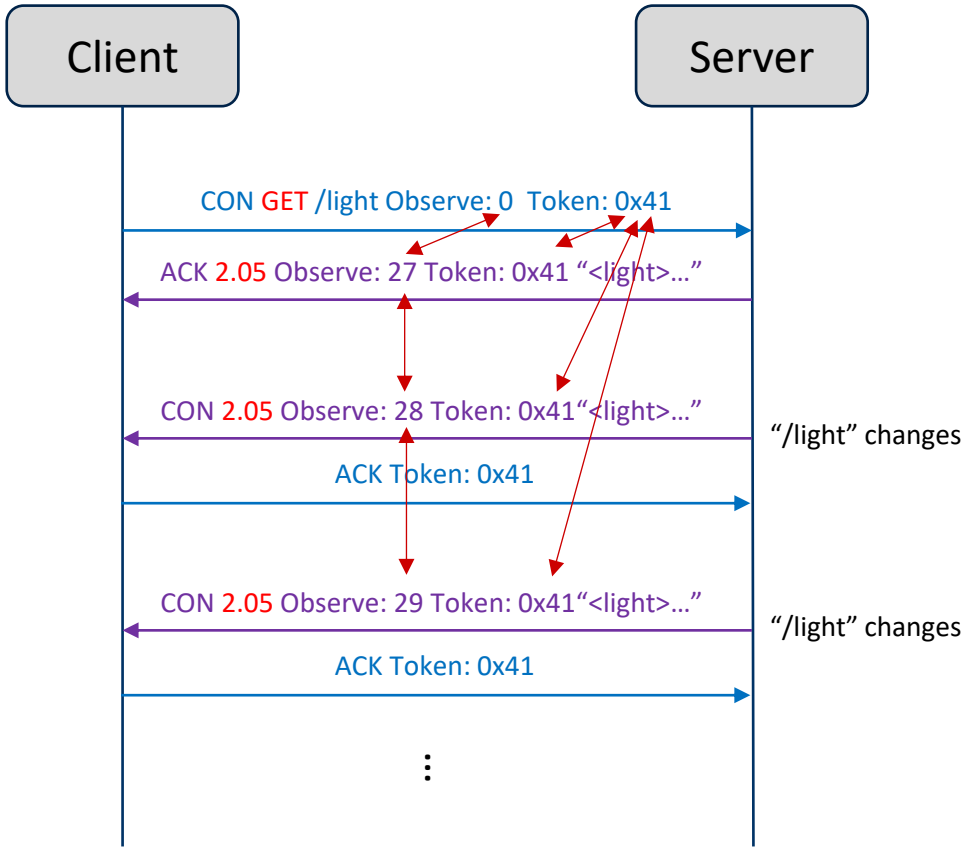
Separate Response



Non-confirmable R&R

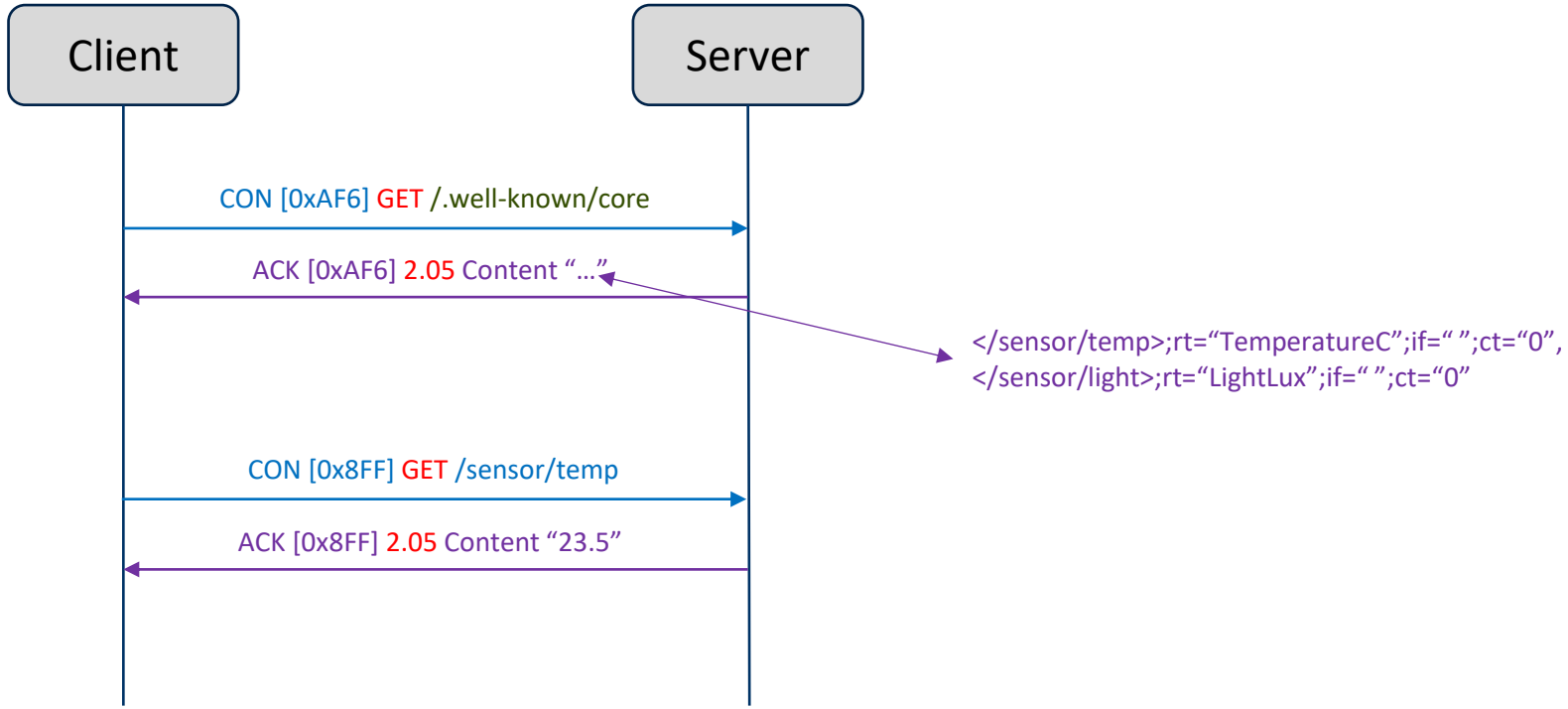


Observation

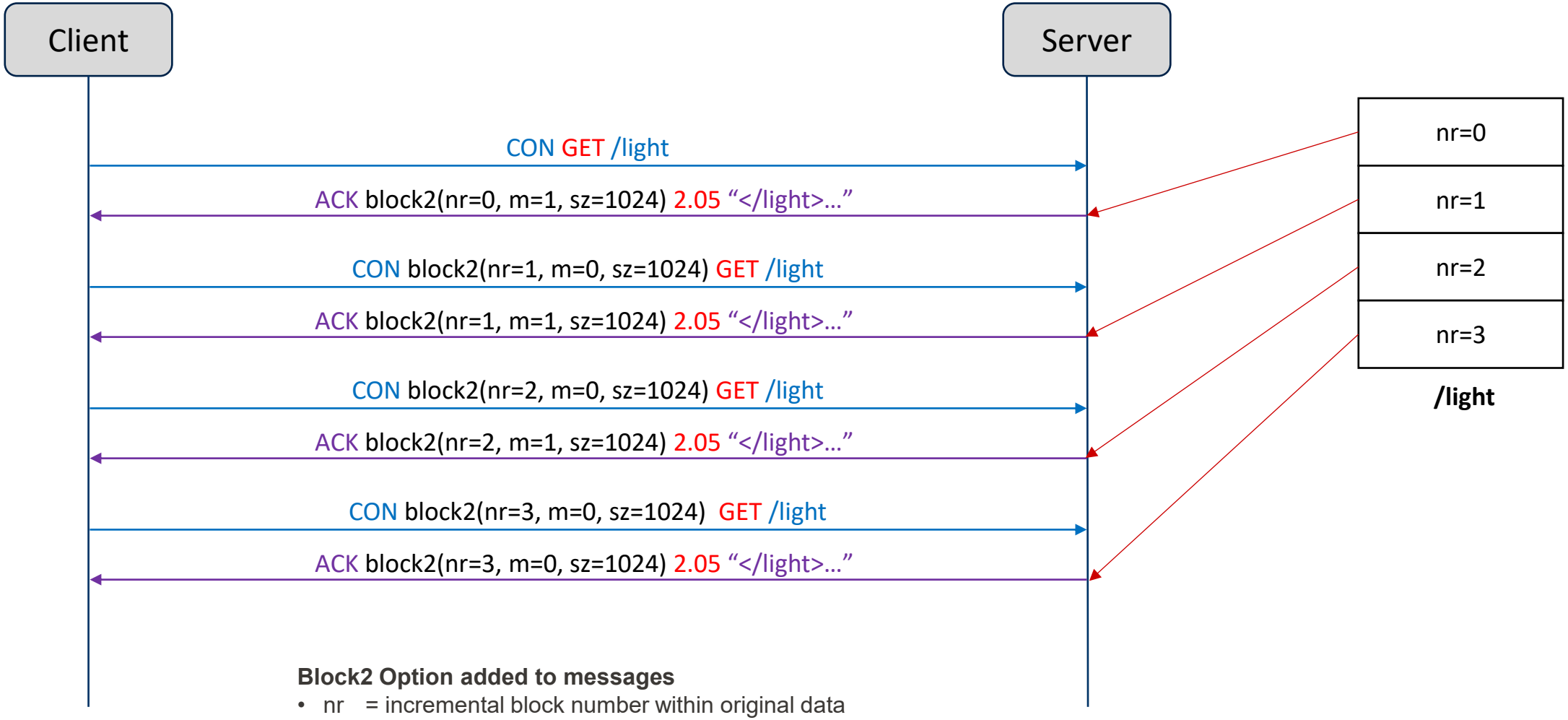


http://tesi.cab.unipd.it/44294/1/TESI.pdf

Resource Discovery



Block Transfer



Block2 Option added to messages

- nr = incremental block number within original data
- m = more blocks flag
- sz = block size

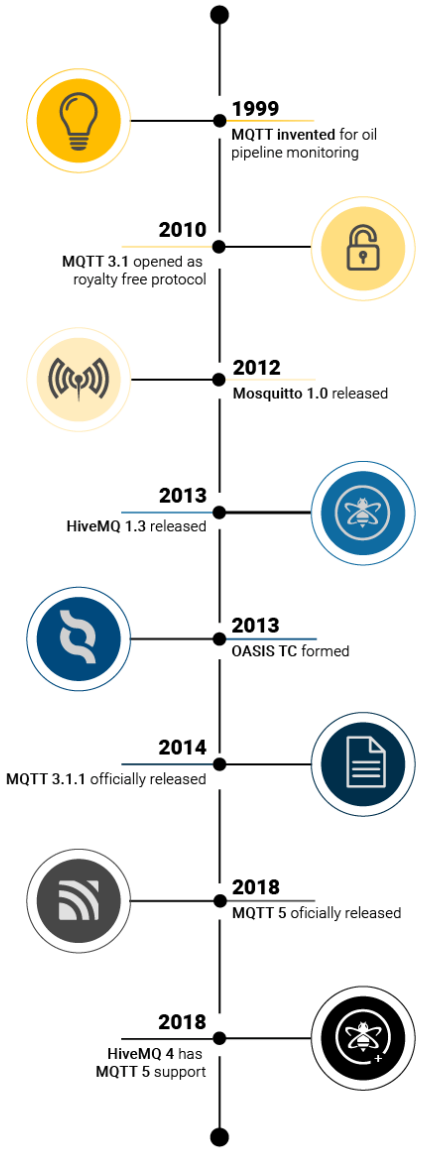
Other CoAP Stuff

- Intermediaries and Caching
- Security - DTLS (TLS/SSL for Datagrams)
- Alternative Transport (e.g. SMS)
- Message Header Options

MQTT

Message Queuing Telemetry Transport (MQTT)

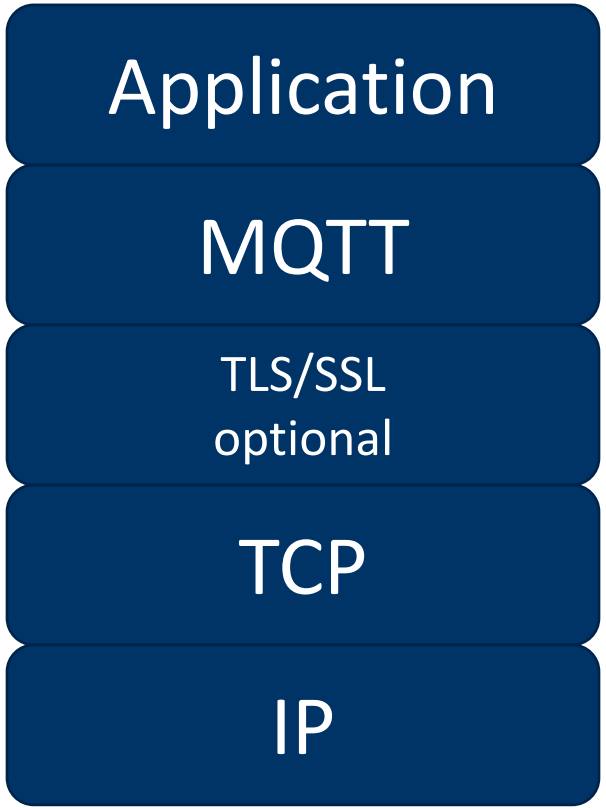
- MQTT was invented by Andy Stanford-Clark (IBM) and Arlen Nipper (Arcom, now Cirrus Link) back in 1999, where their use case was to create a protocol for minimal battery loss and minimal bandwidth connecting oil pipelines over satellite connections. They specified the following goals, which the future protocol should have:
  - Simple to implement
  - Bi-directional asynchronous “push” Communications
  - Provide a QoS Data Delivery
  - Lightweight and Bandwidth Efficient
  - Data Agnostic
  - Continuous Session Awareness



https://www.youtube.com/watch?v=QSwR-JMmNOo&feature=emb_logo
https://www.hivemq.com/blog/mqtt-essentials-part-1-introducing-mqtt/

MQTT

- Built for proprietary embedded systems; now shifting to IoT
- You can send anything as a message; up to 256 MB
- Built for unreliable networks (Continuous Session Awareness)
- Enterprise scale implementations down to hobby projects
- Decouples readers and writers
- Message have a topic, quality of service, and retain status associated with them
- MQTT is a binary based protocol where the control elements are **binary bytes** and not **text strings**.

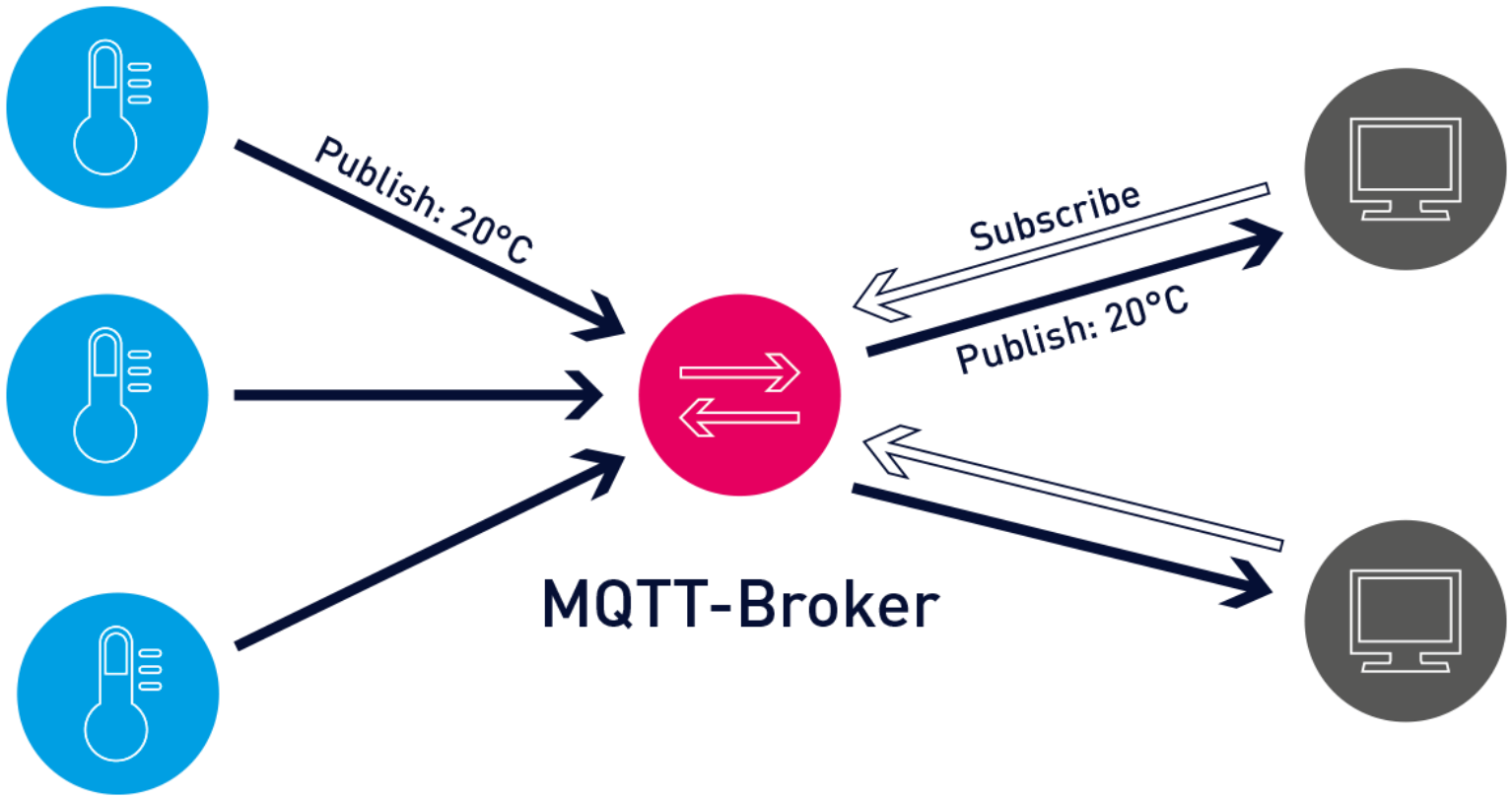


TCP/IP Port: 1883

When running over TLS/SSL, TCP/IP Port 8883
SSL: Secure Socket Layer
TLS: Transport Layer Security*

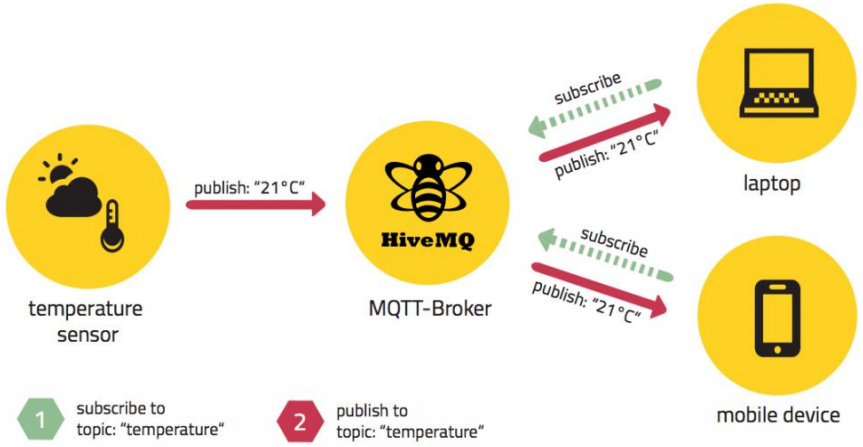
MQTT

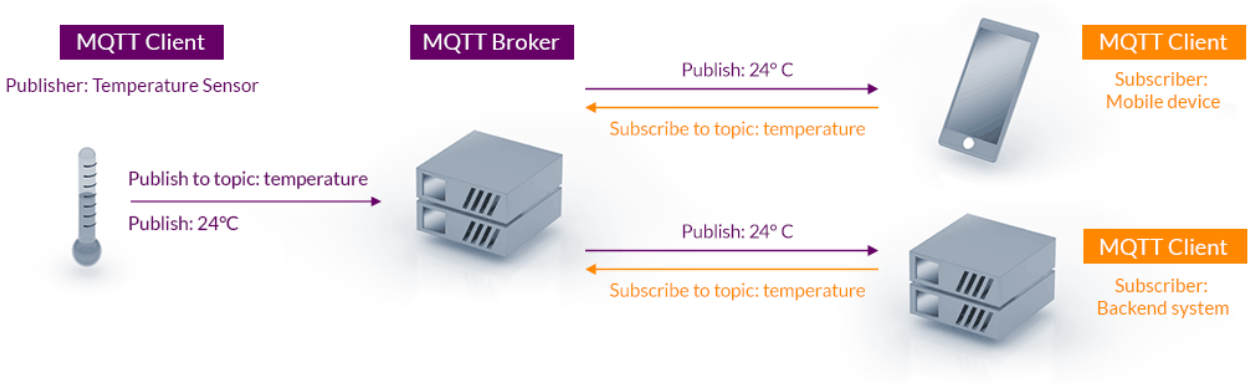
- MQTT consist of three parts:
  - Broker
  - Subscribers
  - Publishers



https://www.paessler.com/it-explained/mqtt

Publish/Subscribe Concept





Decoupled in space and time:

- **Clients** do <u>not</u> need each others IP address
- **Broker**'s IP and port <u>must</u> be known by clients
- **Namespace** hierarchy used for **topic** filtering
- Possible that a published message is never consumed by any subscriber

MQTT: Simple to implement

Connect

Subscribe

Publish

Unsubscribe

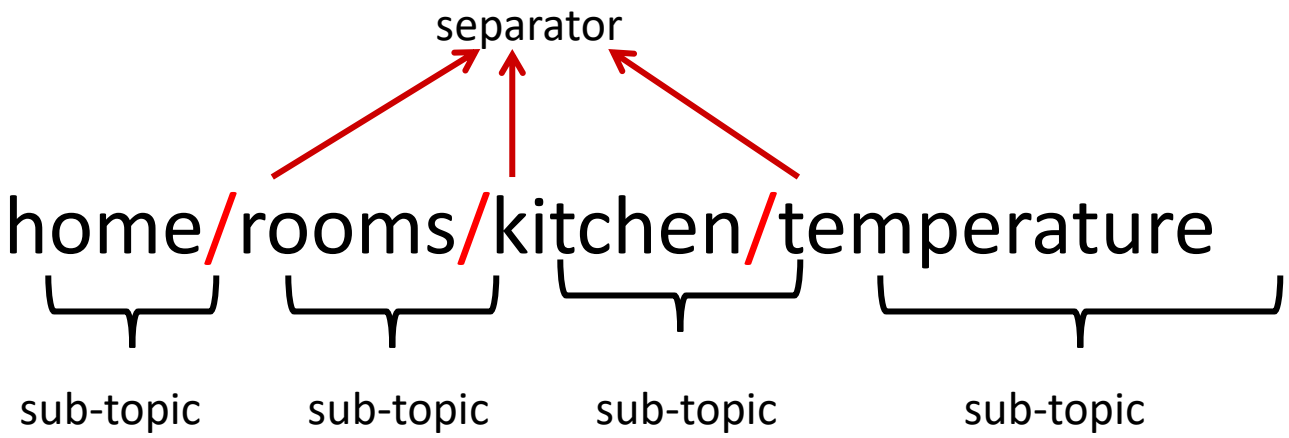
Disconnect

```
client = new Messaging.Client(hostname, port, clientId)
client.onMessageArrived = messageArrived;
client.onConnectionLost = connectionLost;
client.connect({ onSuccess: connectionSuccess });

function connectionSuccess() {
     client.subscribe("planets/earth");
     var msg = new Messaging.Message("Hello world!");
     msg.destinationName = "planets/earth";
     client.publish(msg);
}

function messageArrived(msg) {
     console.log(msg.payloadString);
     client.unsubscribe("planets/earth");
     client.disconnect();
}
```

Eclipse Paho JavaScript MQTT client

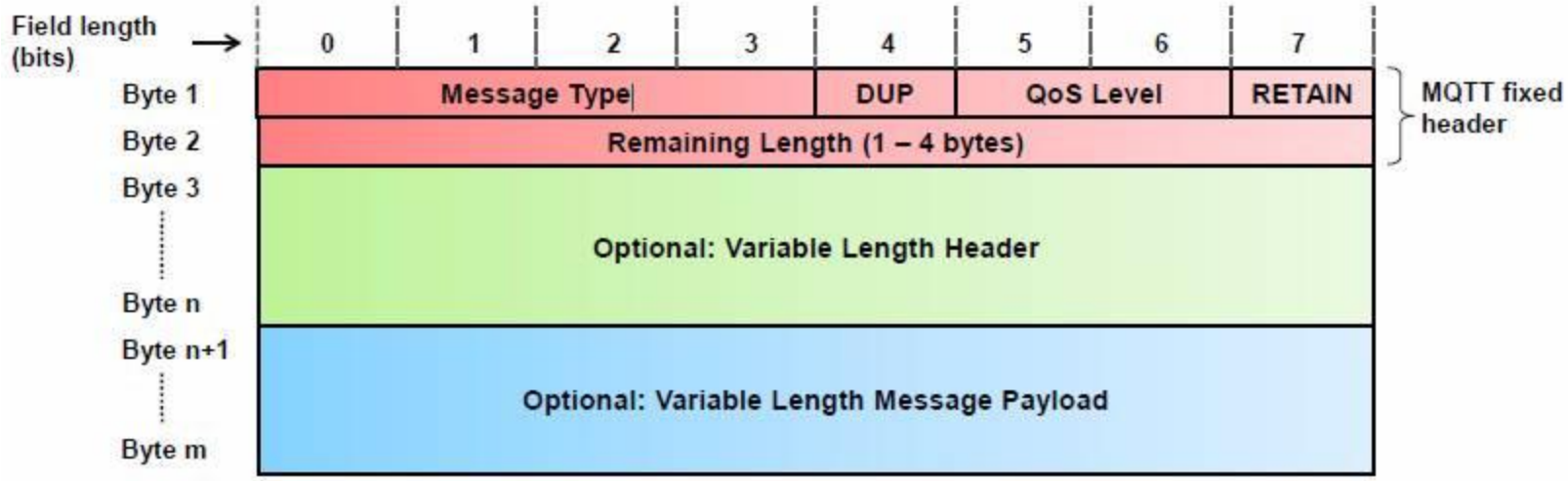
Topics

- Each published data specifies a topic
- Each subscriber subscribed to that topic will receive it
- Topic format:

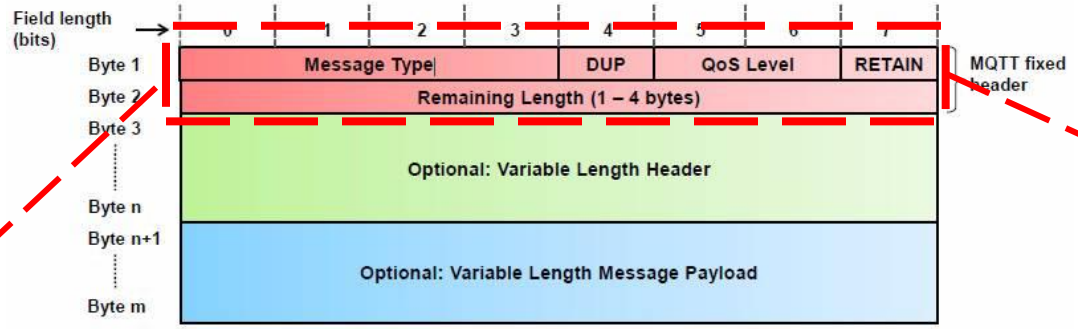


MQTT Message Format

Shortest Message is Two Bytes



MQTT Fixed Header

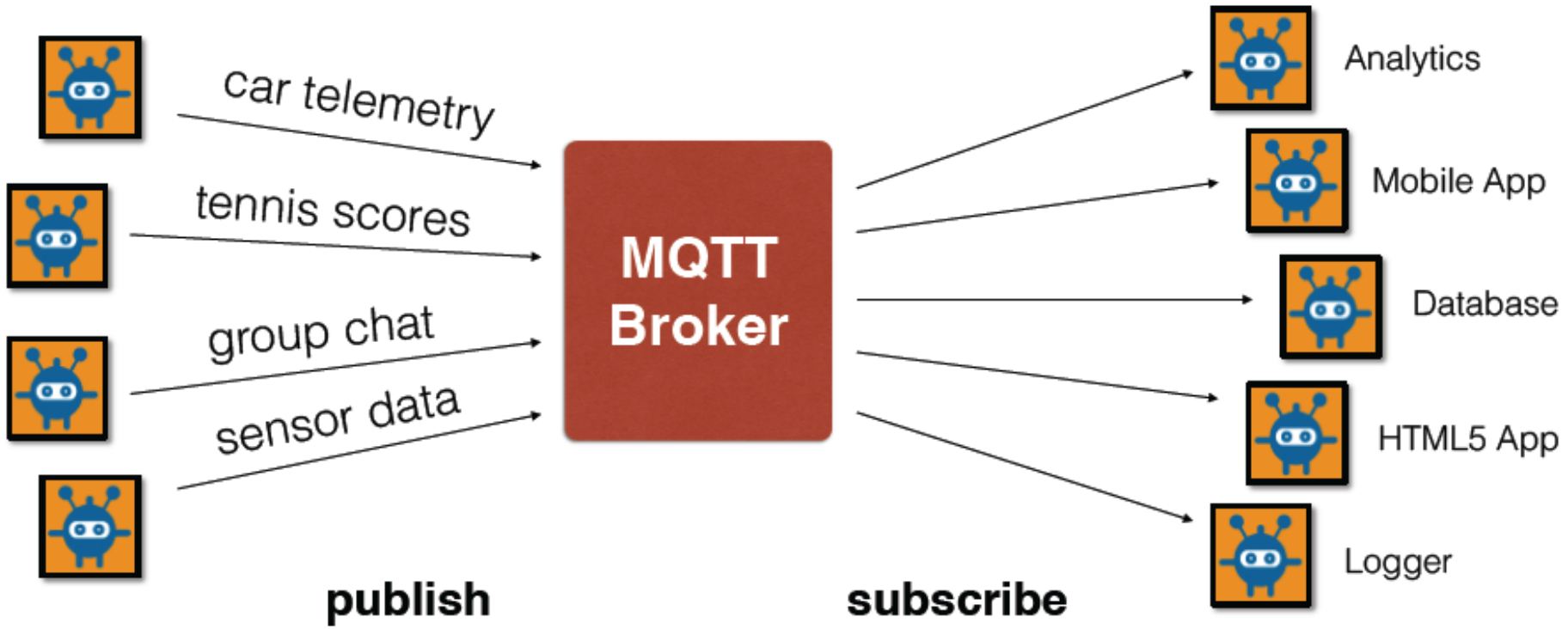


Message fixed header field	Description / Values	
Message Type	0: Reserved	8: SUBSCRIBE
	1: CONNECT	9: SUBACK
	2: CONNACK	10: UNSUBSCRIBE
	3: PUBLISH	11: UNSUBACK
	4: PUBACK	12: PINGREQ
	5: PUBREC	13: PINGRESP
	6: PUBREL	14: DISCONNECT
	7: PUBCOMP	15: Reserved
DUP	Duplicate message flag. Indicates to the receiver that this message may have already been received. 1: Client or server (broker) re-delivers a PUBLISH, PUBREL, SUBSCRIBE or UNSUBSCRIBE message (duplicate message).	
QoS Level	Indicates the level of delivery assurance of a PUBLISH message. 0: At-most-once delivery, no guarantees, «Fire and Forget». 1: At-least-once delivery, acknowledged delivery. 2: Exactly-once delivery. Further details see MQTT QoS.	
RETAIN	1: Instructs the server to retain the last received PUBLISH message and deliver it as a first message to new subscriptions. Further details see RETAIN (keep last message).	
Remaining Length	Indicates the number of remaining bytes in the message, i.e. the length of the (optional) variable length header and (optional) payload. Further details see Remaining length (RL).	

Message Types

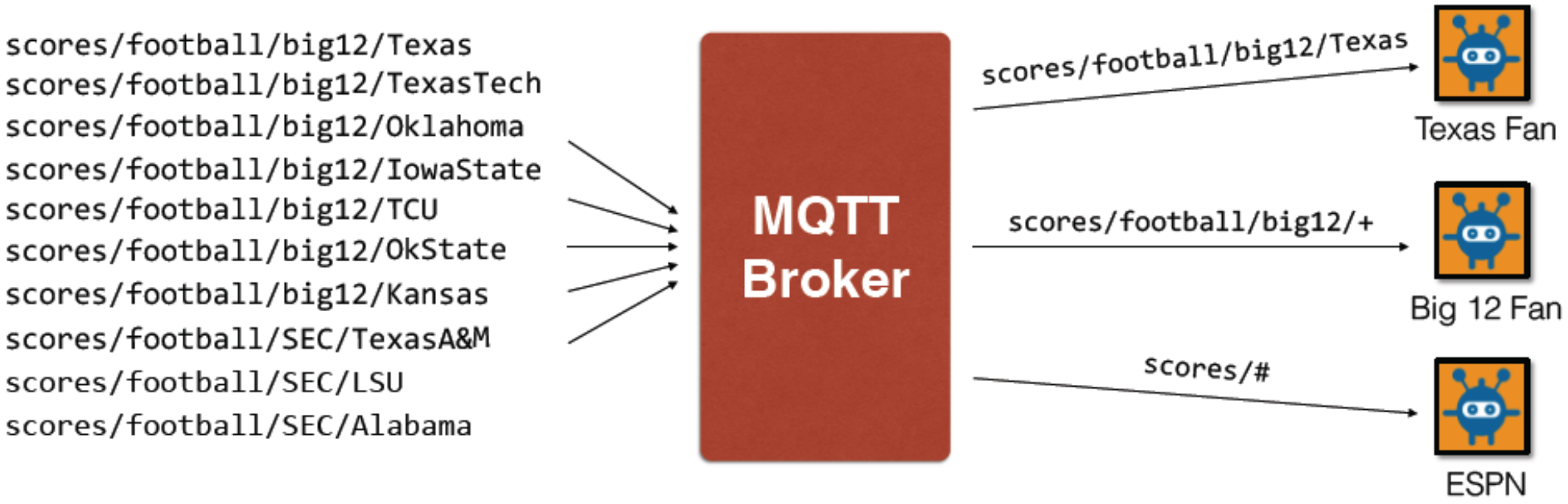
Name	Value	Direction of flow	Description
Reserved	0	Forbidden	Reserved
CONNECT	1	Client to Server	Client request to connect to Server
CONNACK	2	Server to Client	Connect acknowledgment
PUBLISH	3	Client to Server or Server to Client	Publish message
PUBACK	4	Client to Server or Server to Client	Publish acknowledgment
PUBREC	5	Client to Server or Server to Client	Publish received (assured delivery part 1)
PUBREL	6	Client to Server or Server to Client	Publish release (assured delivery part 2)
PUBCOMP	7	Client to Server or Server to Client	Publish complete (assured delivery part 3)
SUBSCRIBE	8	Client to Server	Client subscribe request
SUBACK	9	Server to Client	Subscribe acknowledgment
UNSUBSCRIBE	10	Client to Server	Unsubscribe request
UNSUBACK	11	Server to Client	Unsubscribe acknowledgment
PINGREQ	12	Client to Server	PING request
PINGRESP	13	Server to Client	PING response
DISCONNECT	14	Client to Server	Client is disconnecting
Reserved	15	Forbidden	Reserved

MQTT: pub/sub decouples sender from receiver



https://www.slideshare.net/BryanBoyd/mqtt-austin-iot-meetup

MQTT: allows wildcard subscription



single level wildcard: **+** multi-level wildcard: **#**

get all the bedroom data → **myhome/bedroom/+**
get the temperature data of three rooms → **myhome/+/temperature**
get all the data → **myhome/#** or **#**

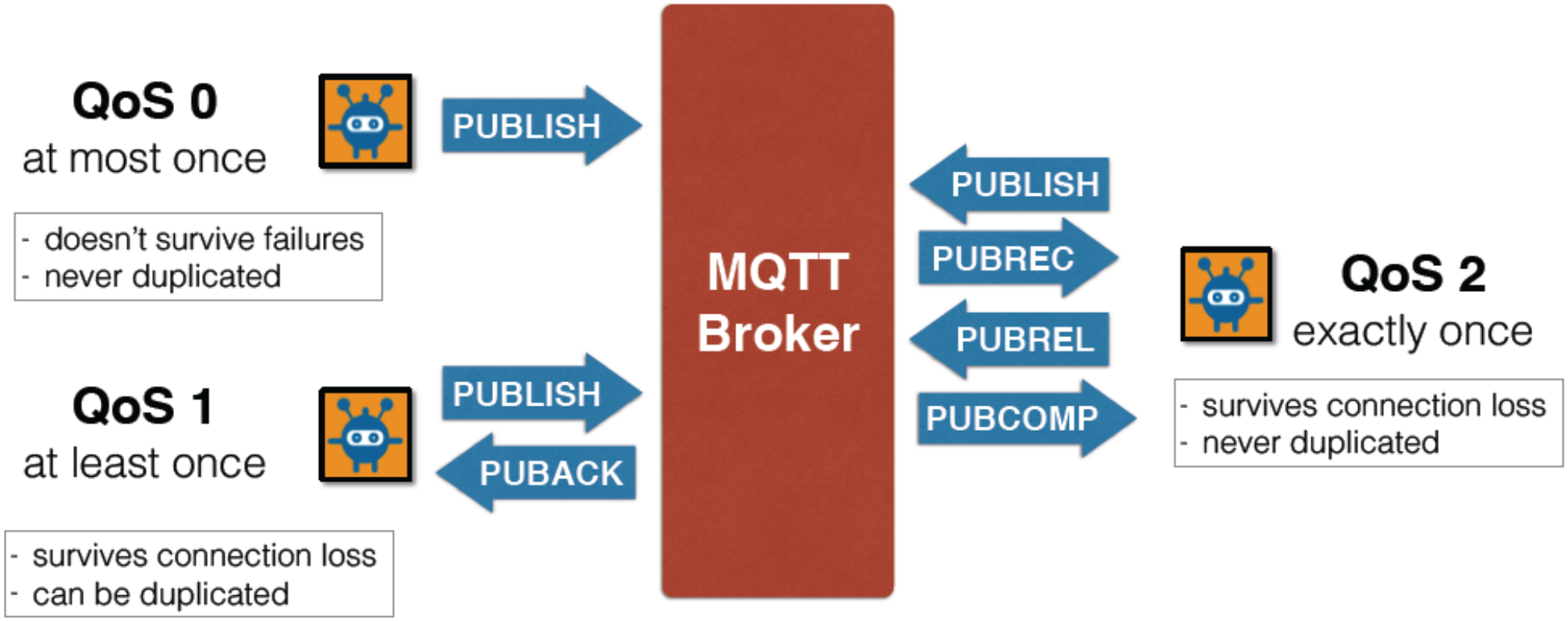
https://www.slideshare.net/BryanBoyd/mqtt-austin-iot-meetup

Best Practices

- Do not use topics beginning with $
- Never use a leading forward slash: /myhome/groundfloor/livingroom
- Embed a unique identifier or the Client Id into the topic: client1/status
- Don’t forget extensibility: think about future topic names
- Use specific topics, not general ones:
  - myhome/livingroom/temperature
  - myhome/livingroom/brightness
  - myhome/livingroom/humidity
- Don’t subscribe to #
- Never use spaces in a topic
- Keep the topic short and concise
- Use only ASCII characters; avoid non-printable characters

https://www.hivemq.com/blog/mqtt-essentials-part-5-mqtt-topics-best-practices/

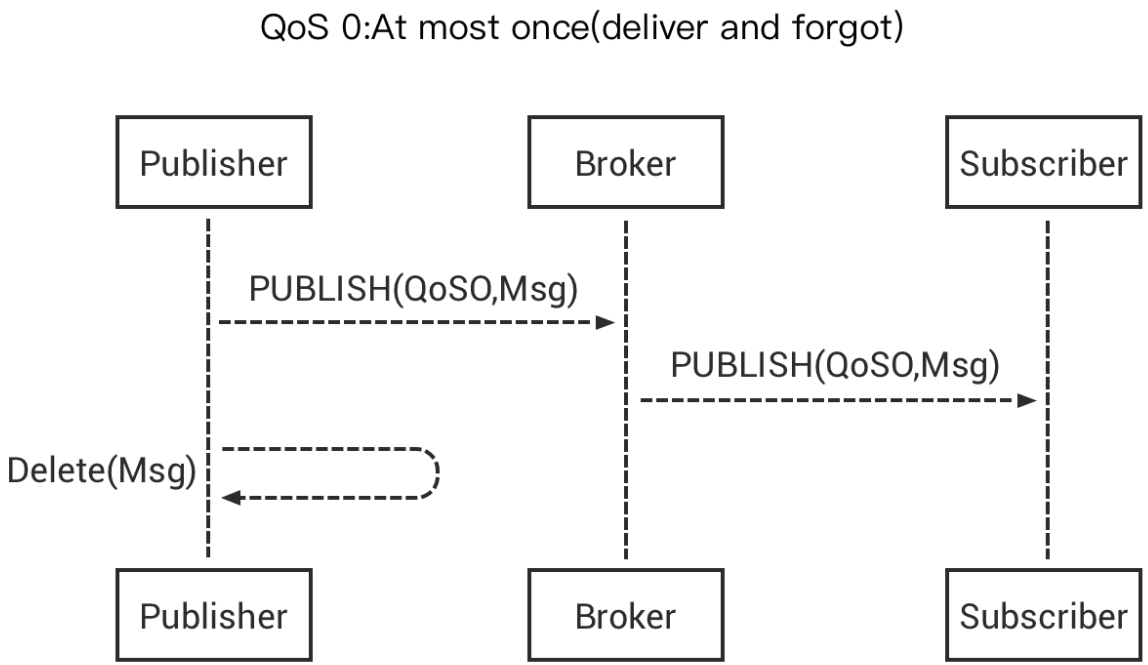
MQTT: reliable messaging



https://www.slideshare.net/BryanBoyd/mqtt-austin-iot-meetup

Publishing “QoS 0” – Unreliable

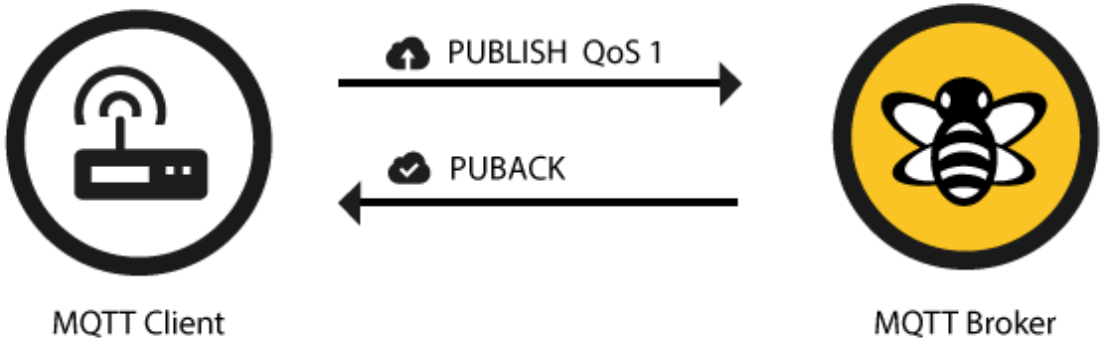
- 0 – unreliable (aka “**at most once**”)
  - OK for continuous streams, least overhead (1 message)
  - “Fire and forget”
  - TCP will still provide reliability

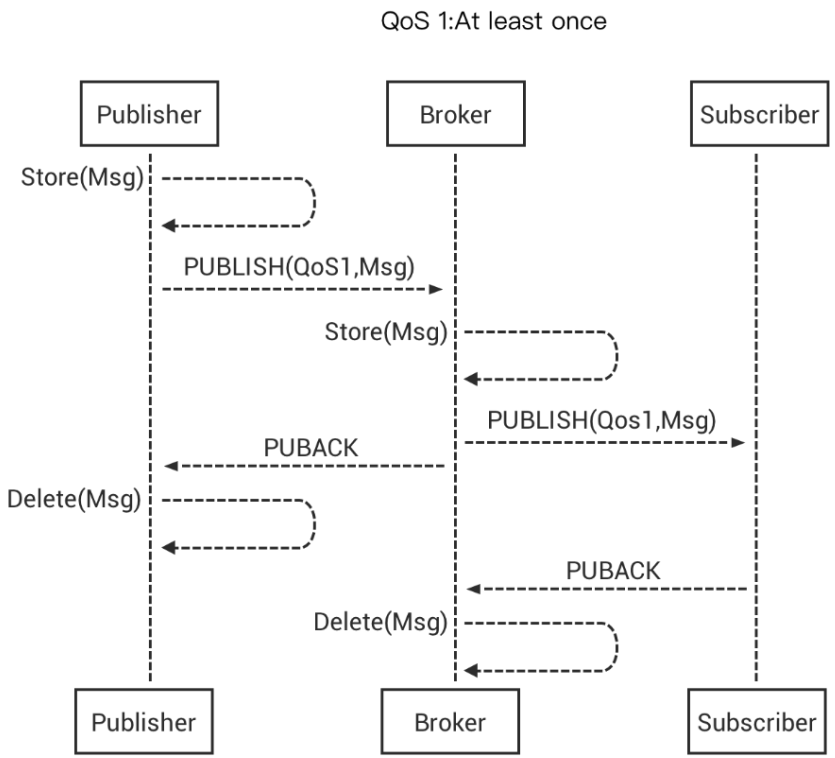


https://www.emqx.com/en/blog/introduction-to-mqtt-qos

Publishing “QoS 1” – At Least Once

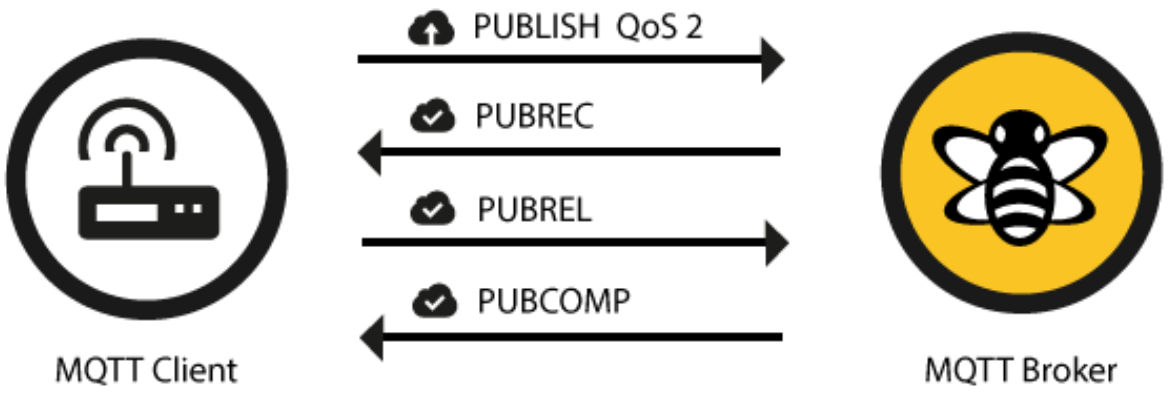
- 1 – delivery “**at least once**” (duplicates possible)
  - Used for alarms – more overhead (2 messages)
  - Contains message ID (to match with **ACK**ed message)

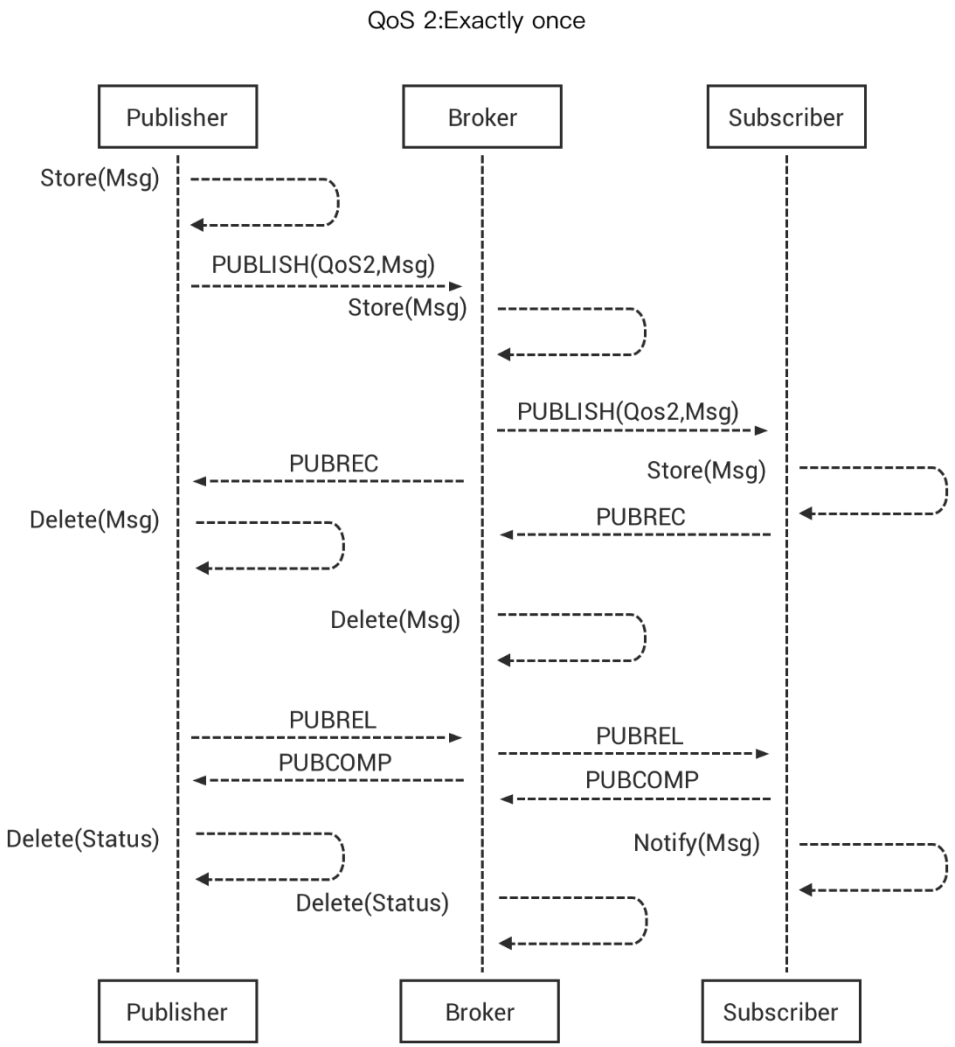




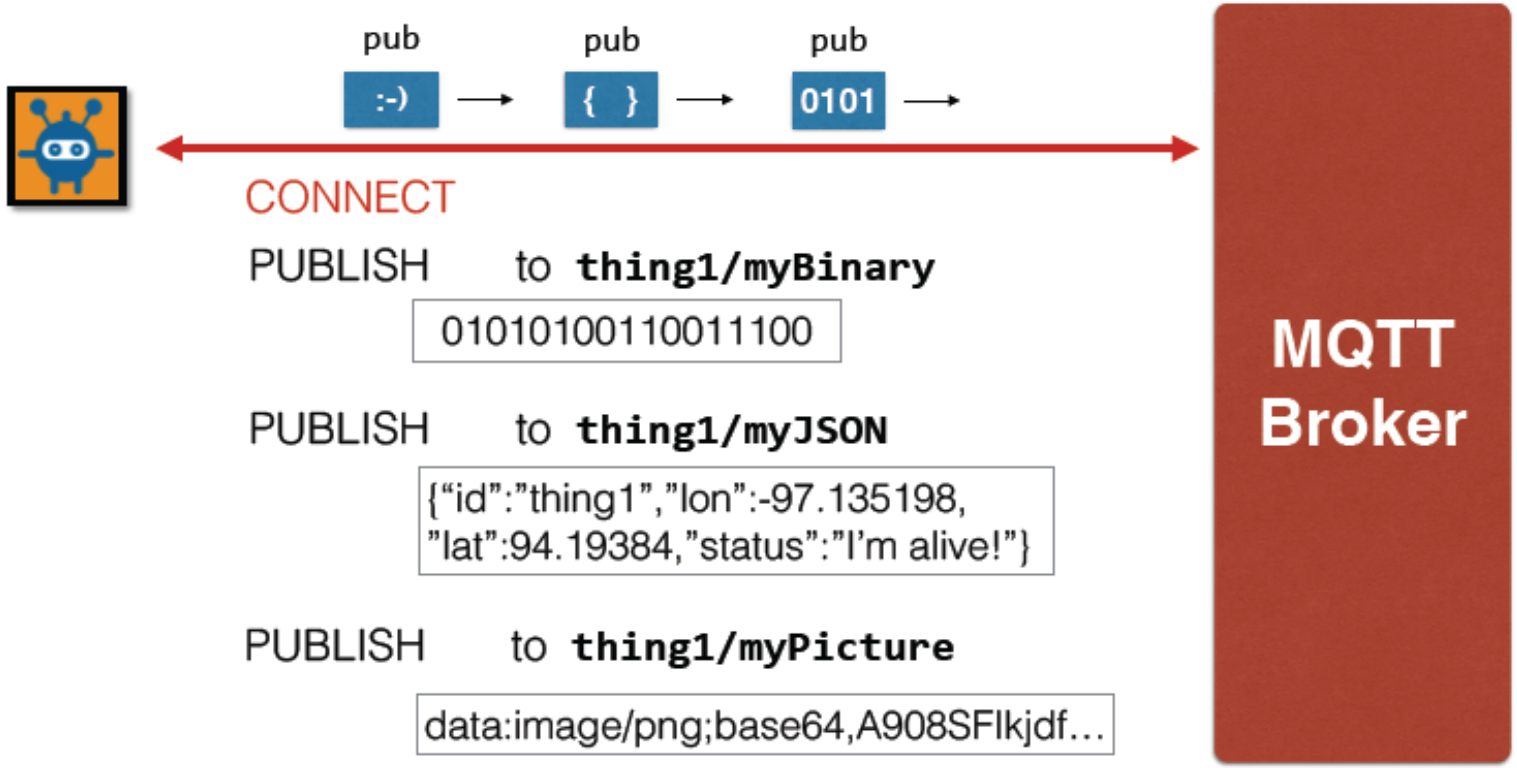
Publishing “QoS 2” – Exactly Once

- 2 – delivery “**exactly once**”
  - Utmost reliability is important – most overhead (4 messages) and slowest

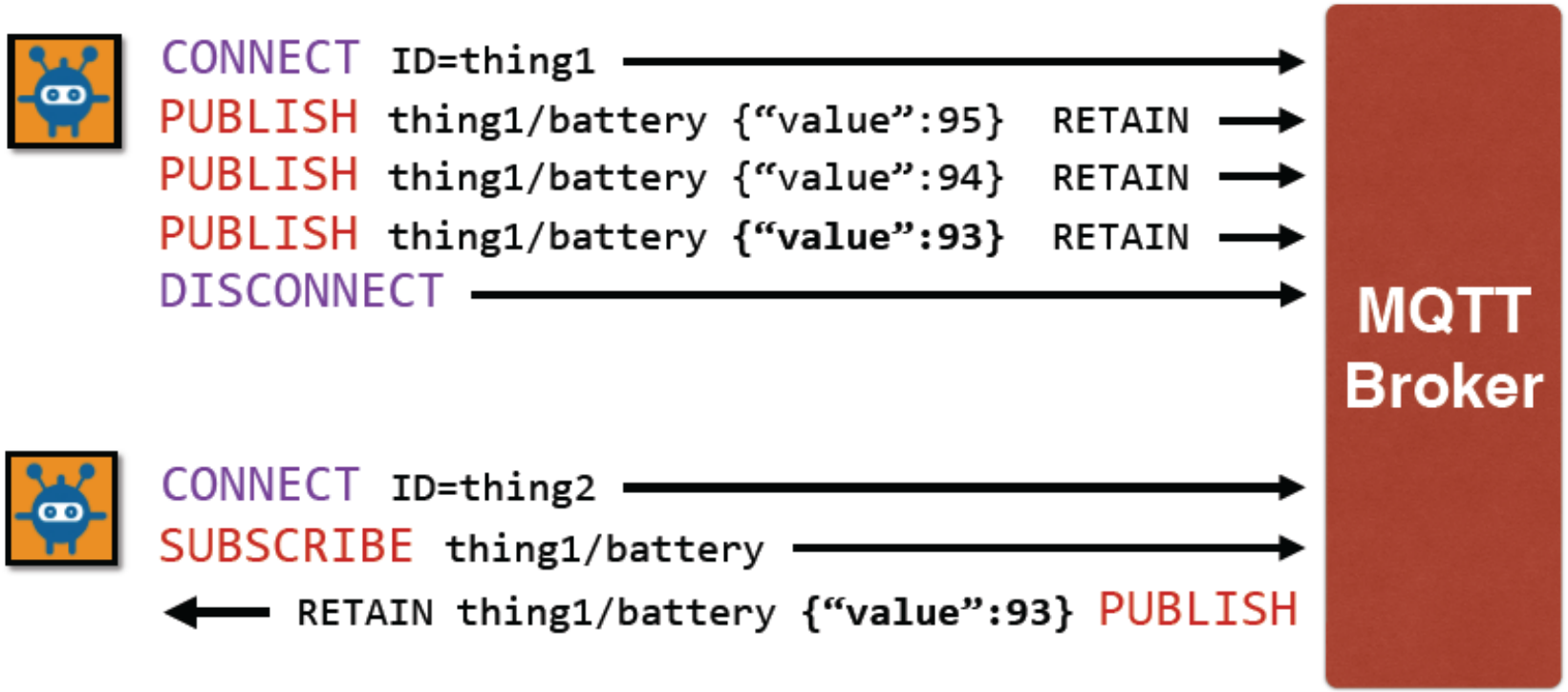




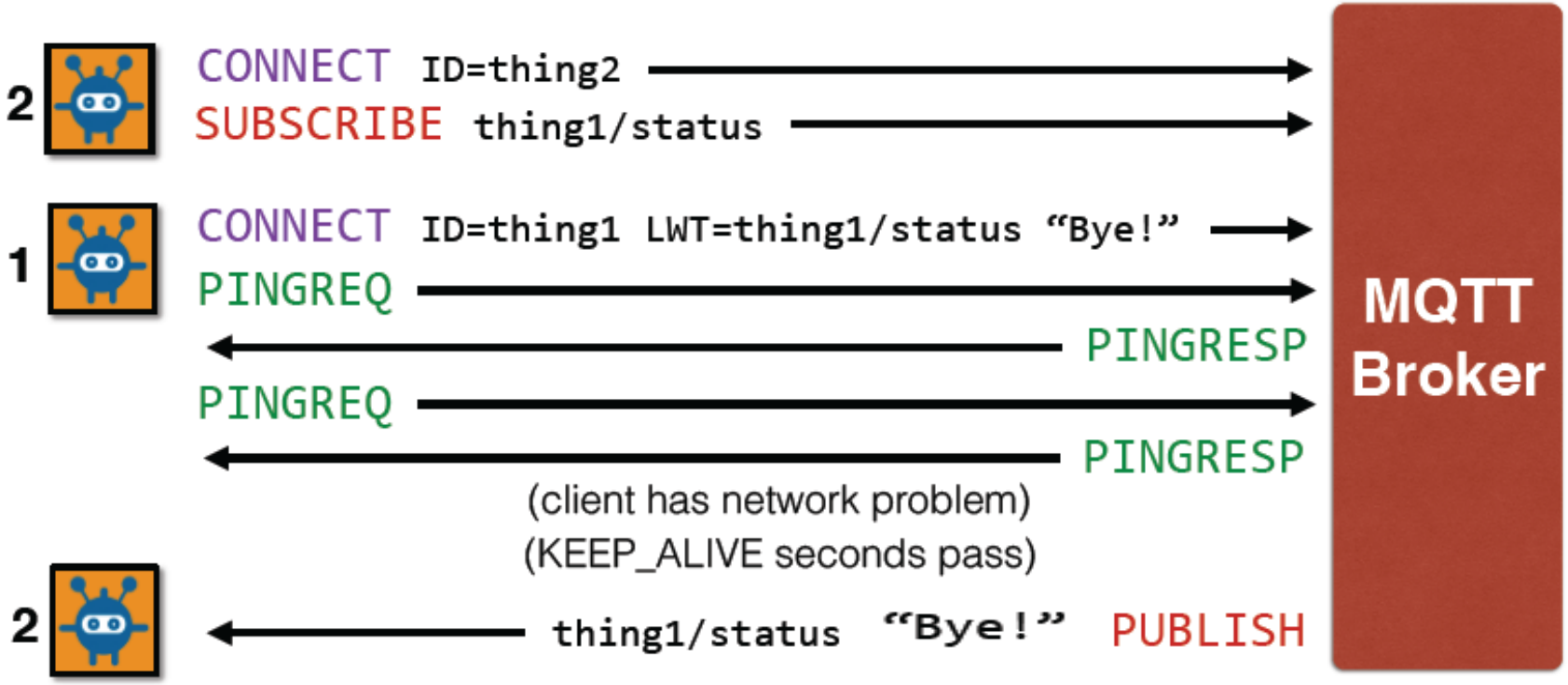
MQTT: Flexible Payload



MQTT: Retain Message (state retention)



MQTT: Last Will and Testament (LWT)



Paho-MQTT (Python-client Example)

```
#import the client
import paho.mqtt.client as mqtt

#create new instance
client = mqtt.Client("P1")

#connecting to broker (host, port, keepalive, bind_address)
client.connect("iot.eclipse.org“, 1883, 60, “192.168.1.184”)

#Subscribing to topic “house/bulb1” with QoS=1
client.subscribe(“house/bulb1”, 1)
client.subscribe([(“house/bulb3”,2),(“house/bulb4”,1),(“house/bulb5”,0)])

#Publishing message to topic “house/bulb1” with “OFF” payload and QoS of 1 with retain=FALSE
client.publish(“house/bulb1”, “OFF”, 1, FALSE)
```

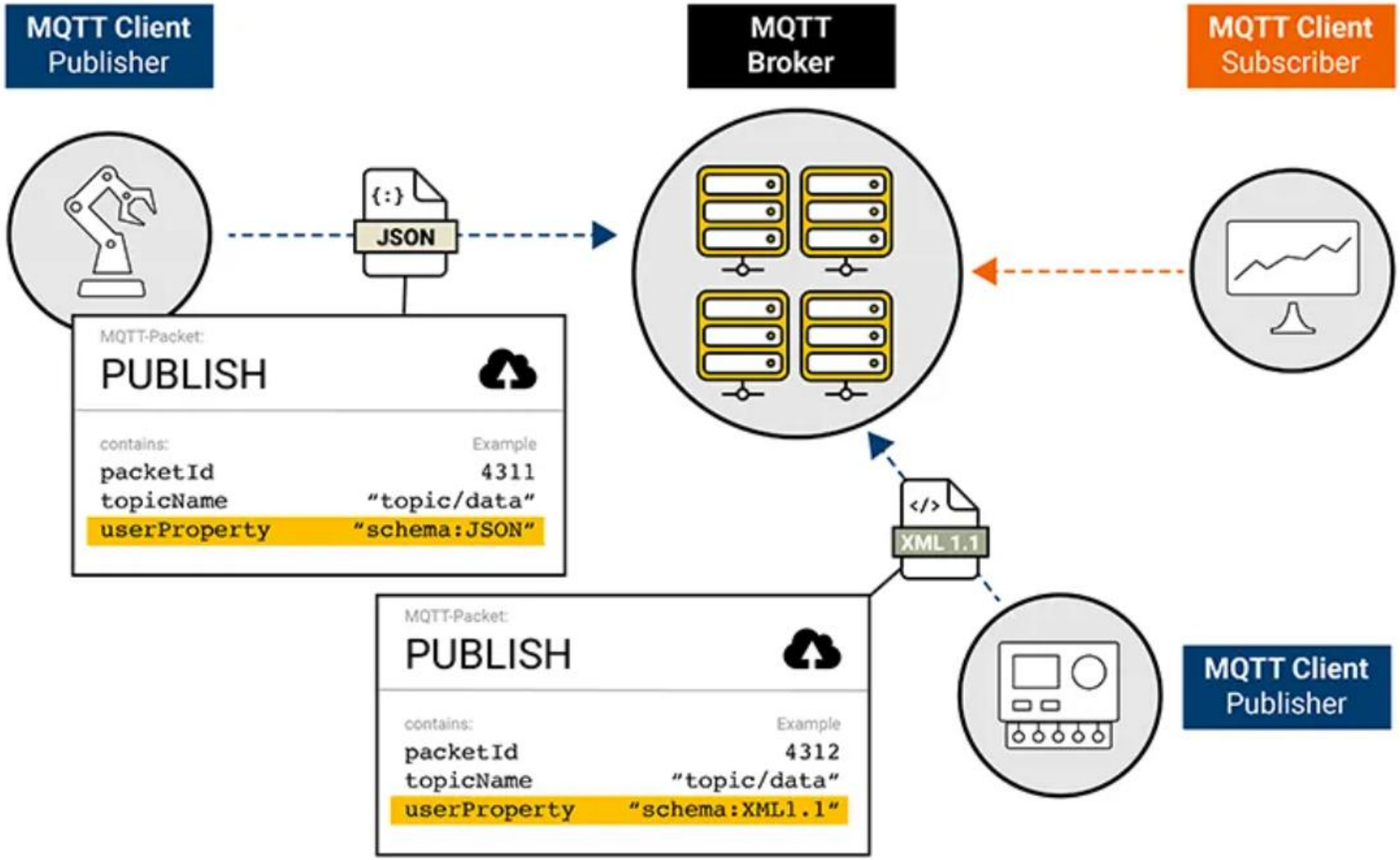
subscribing to **multiple topics**

http://www.steves-internet-guide.com/into-mqtt-python-client/

MQTT 5 – Whats New

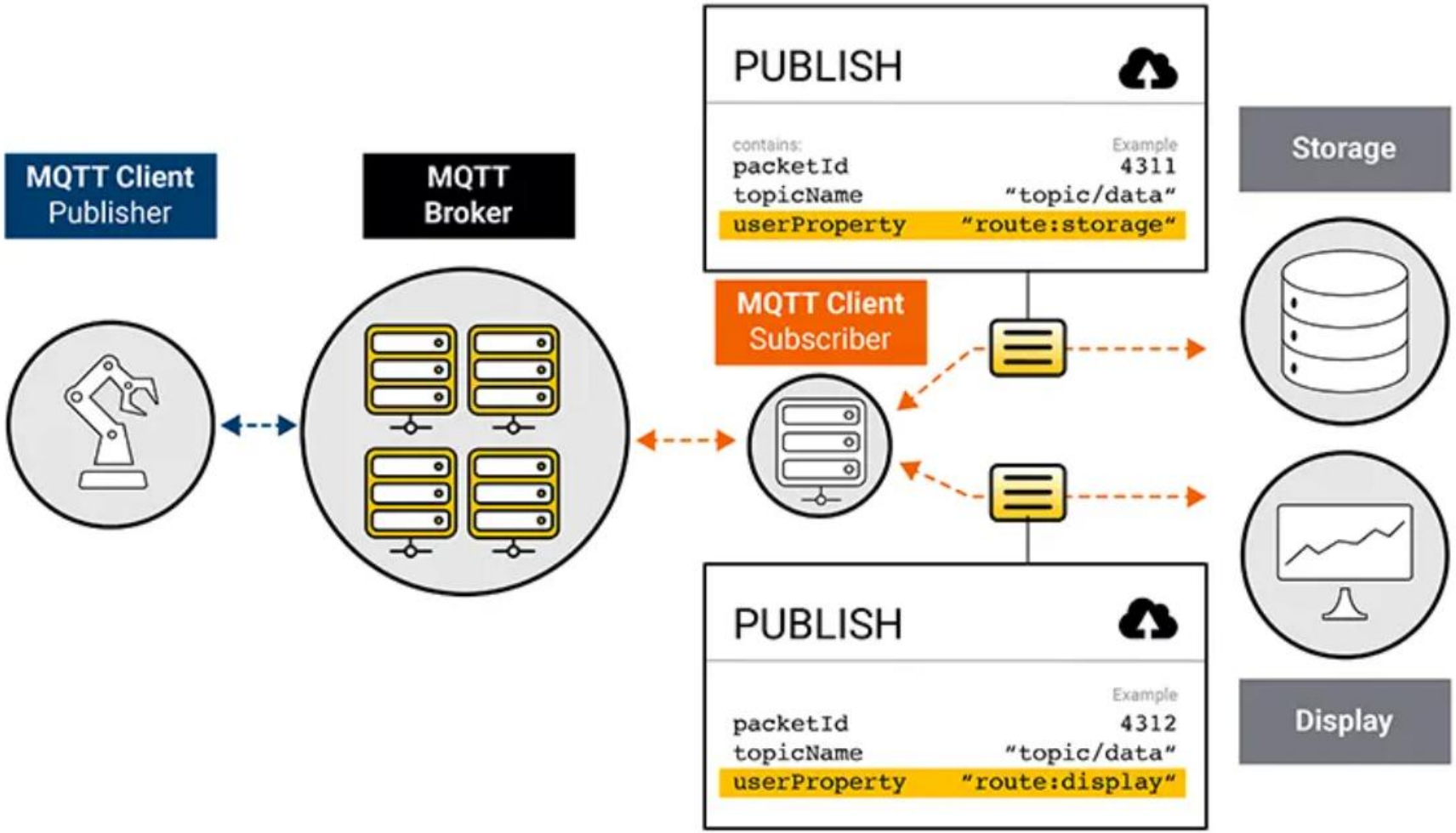
- **User Properties**
- **Shared Subscriptions**
- **Payload Format**
- **Request-Response**
- **Topic Alias**
- **Enhanced Authentication**
- **Flow Control**
- Expiry Interval
  - Message
  - Session
- 128 Reason Codes (19 in MQTT 3.1.1) and Reason String
- Improved Client Feedback – List of supported features
- Prohibition of retransmitting MQTT messages for healthy TCP connections
- Passwords without usernames for authentication

User Properties - What



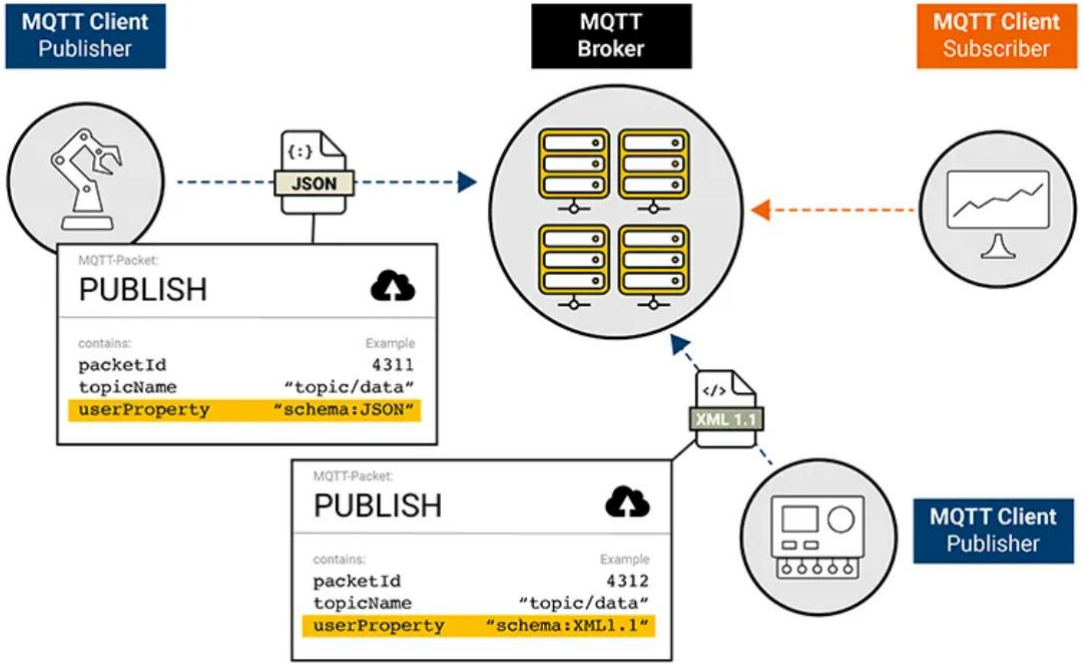
Resource: https://www.hivemq.com/blog/mqtt5-essentials-part6-user-properties/

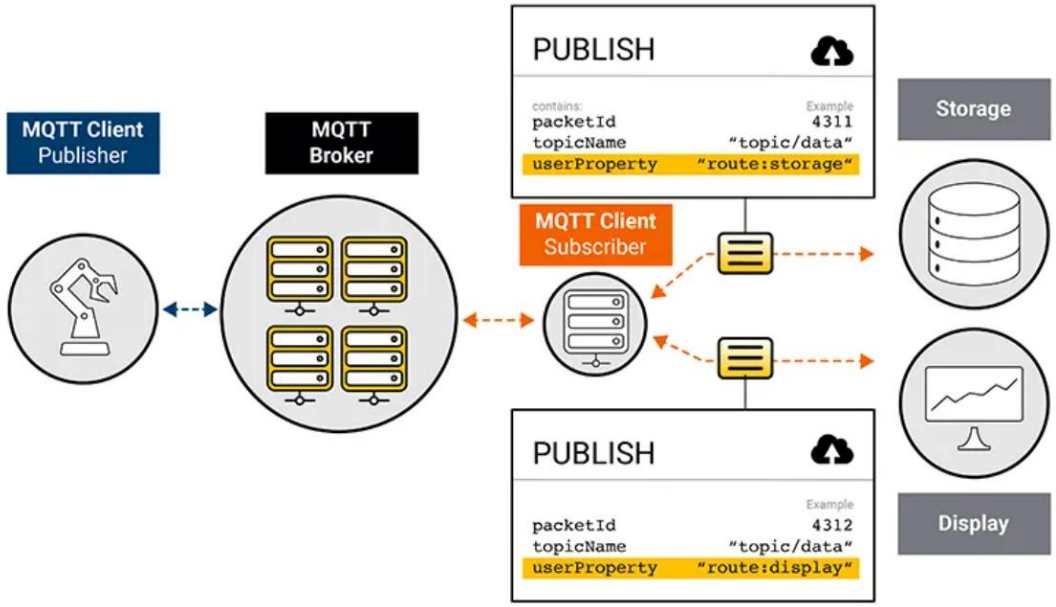
User Properties - Why



Resource: https://www.hivemq.com/blog/mqtt5-essentials-part6-user-properties/

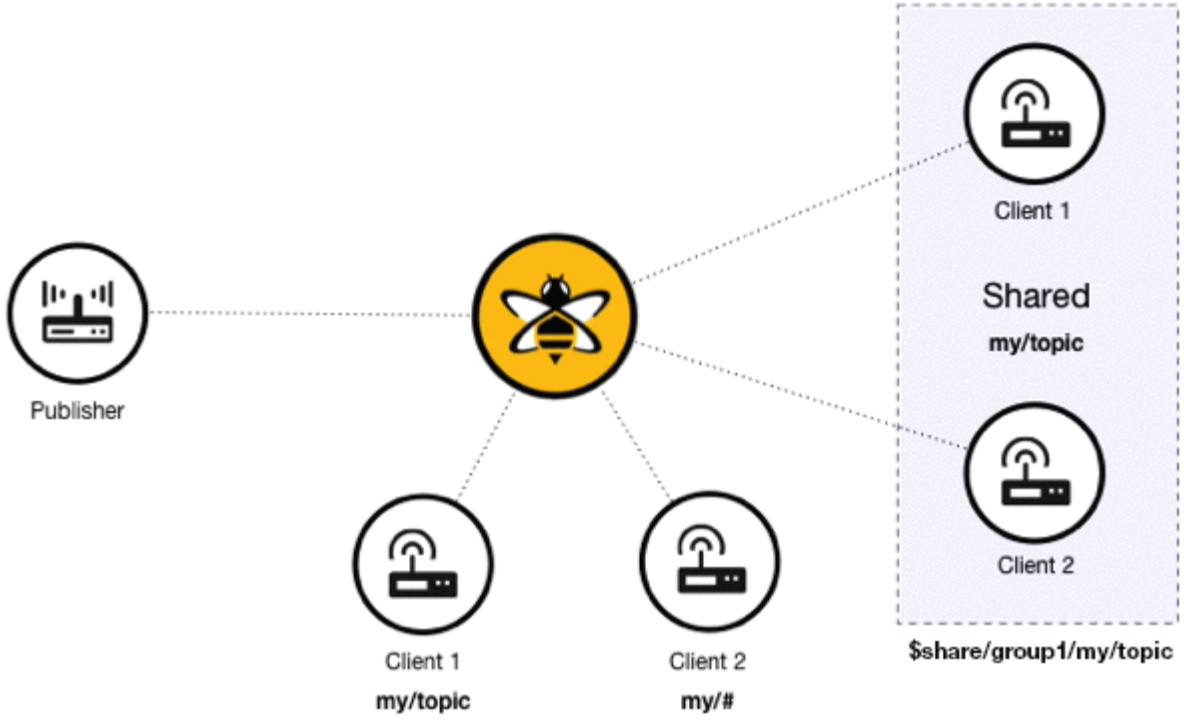
User Properties – Why not just put it all in the payload





Resource: https://www.hivemq.com/blog/mqtt5-essentials-part6-user-properties/

Shared Subscriptions



$share/GROUPID/TOPIC

The shared subscription consists of 3 parts:

- A static shared subscription identifier ($share)
- A group identifier
- The actual topic subscriptions (may include wildcards)

Example for a subscriber: **$share/my-shared-subscriber-group/myhome/groundfloor/+/temperature**

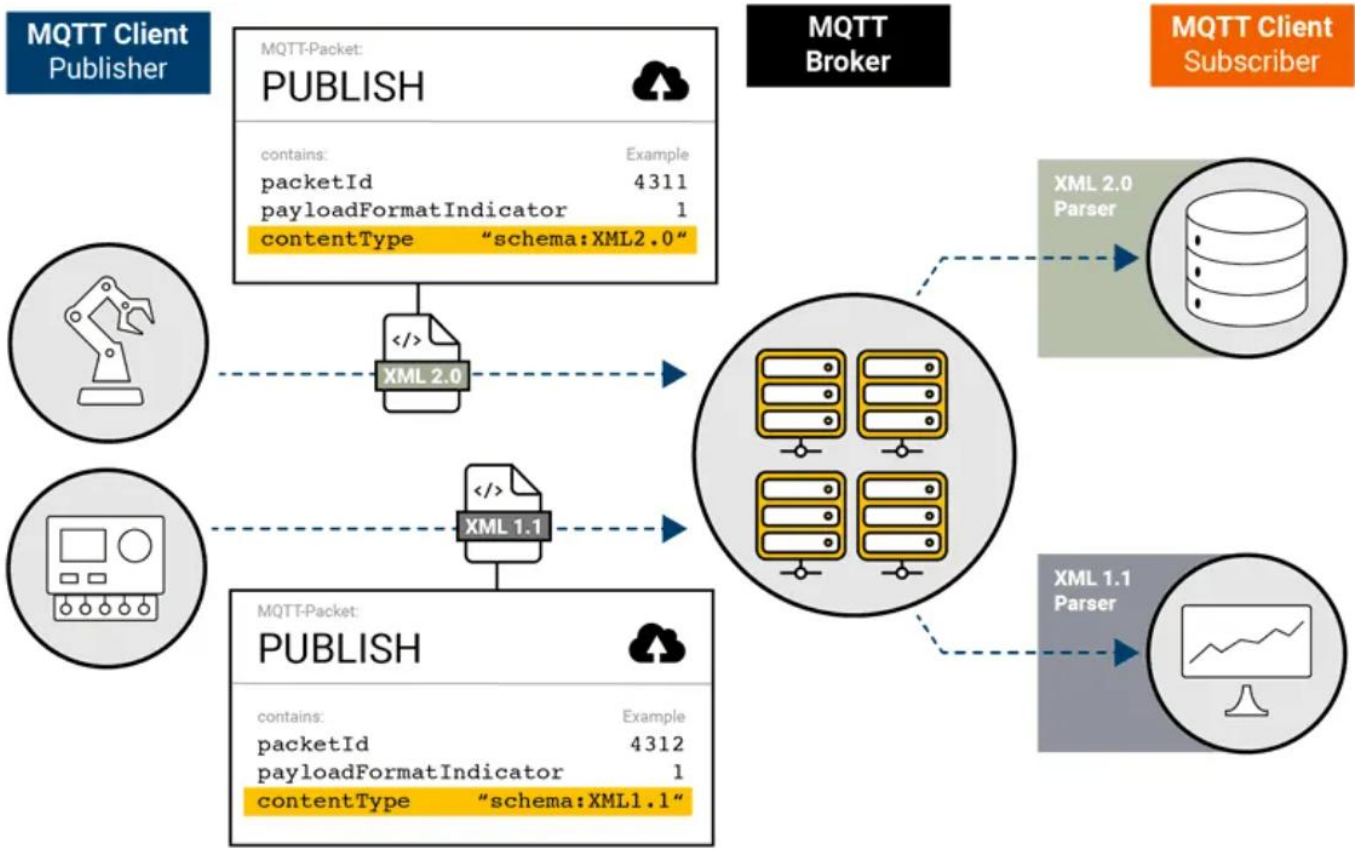
Resource: https://www.hivemq.com/blog/mqtt5-essentials-part7-shared-subscriptions/

Payload Format Indicator & Content Type

MQTT-Packet:
PUBLISH

packetId	"client1"
topicName	"client1/status"
payloadFormatIndicator	0
contentType	SolarPanelSchemaV1.0
qos	1
retainFlag	true
payload	"online"

- 0 indicates an "unspecified byte stream"
- 1 represents a "UTF-8 encoded payload."
  - MIME-like content type descriptor
- When the Payload Format Indicator isn't provided, it automatically defaults to 0.



Resource: https://www.hivemq.com/blog/mqtt5-essentials-part8-payload-format-description/

Payload Format Indicator & Content Type (Example)

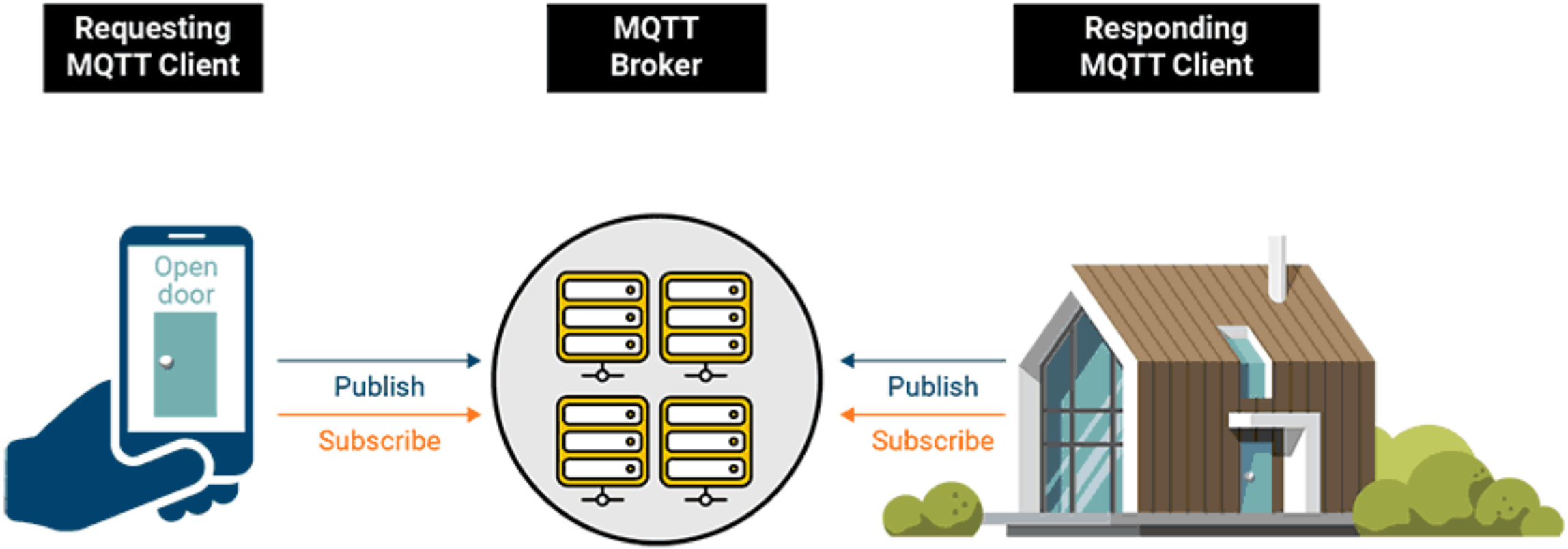
Temperature reading (JSON)

```
Topic: greenhouse/telemetry
Payload: { "temp": 25.1 }
Payload Format: 1
Content Type: application/json
```

Image frame from a camera (binary JPEG)

```
Topic: greenhouse/camera
Payload: <raw JPEG bytes>
Payload Format: 0
Content Type: image/jpeg
```

Request-Response



Request-Response (example)

Client Request

```
Topic: sensors/request/deviceA
Response Topic: sensors/response/client1
Correlation Data: [42]
```

Device Response

```
Topic: sensors/response/client1
Correlation Data: [42]
Payload: "Temperature = 21.4"
```

Topic Alias

MQTT-Packet:

PUBLISH

contains:	Example
packetID	4313
topicName	"data/europe/germany/south/bavaria/munich/schwabing/box-32543y/junction/consumption"
	321
topicAlias	1
qos	756
payload	

- Payload are relatively small
- Topics are relatively long
- Message throughput is relatively high

Resource: https://www.hivemq.com/blog/mqtt5-essentials-part10-topic-alias/

Topic Alias (example)

Initial Publish (introducing alias)

PUBLISH:

- Topic Name: sensors/room1/temp
- Topic Alias: 1
- Payload: 22.7

Subsequent Publish (using alias only)

PUBLISH:

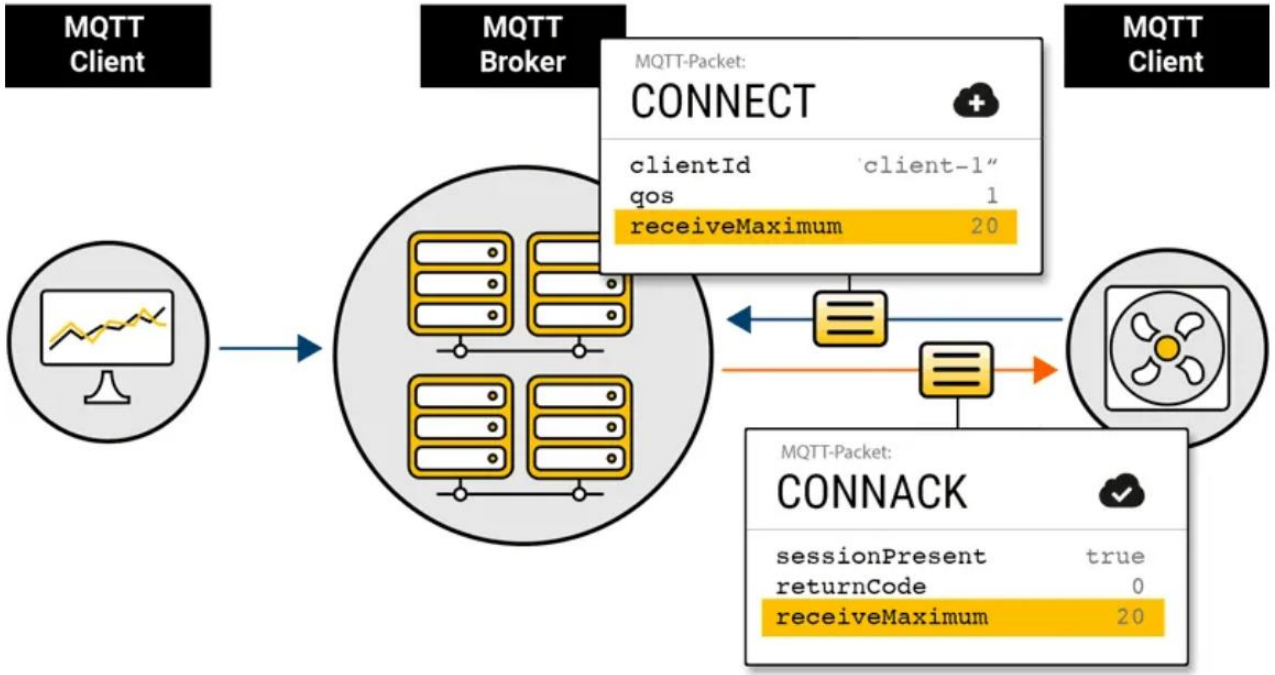
- Topic Name: (empty)
- Topic Alias: 1
- Payload: 22.8

Future Publish (remapping alias)

PUBLISH:

- Topic Name: sensors/room3/humidity
- Topic Alias: 1
- Payload: 21.4

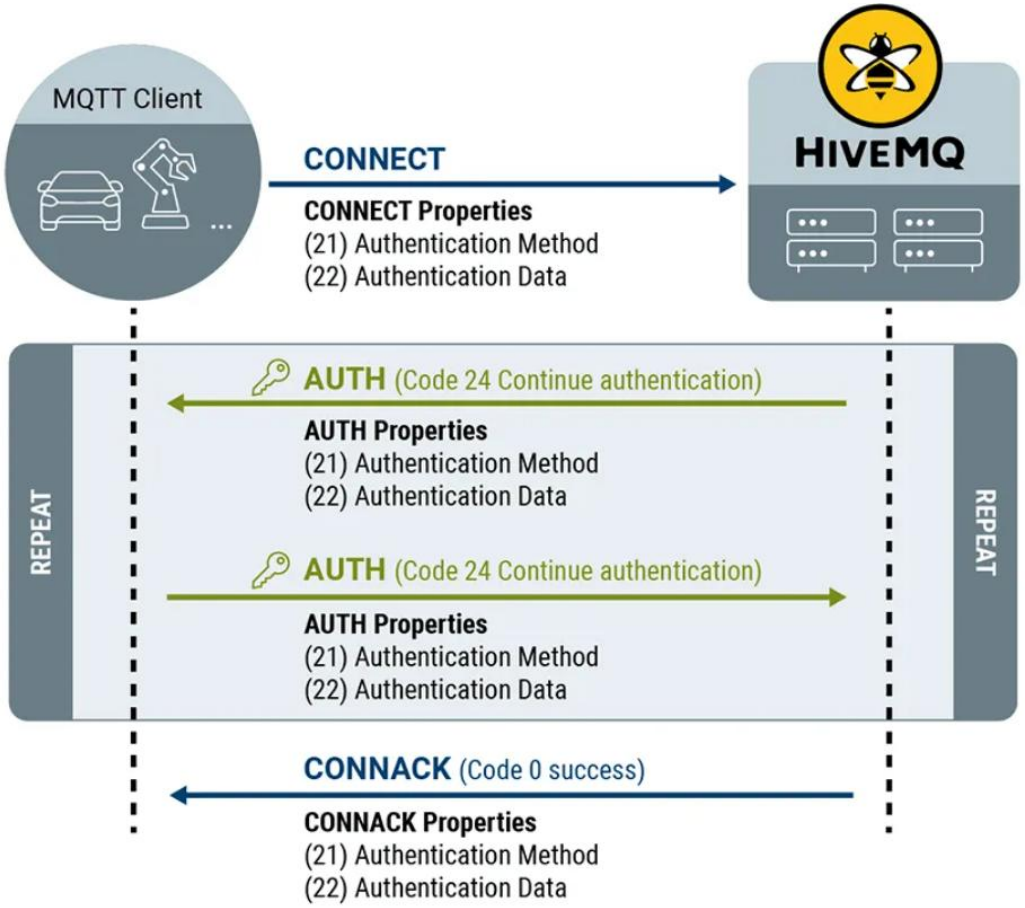
Flow Control



- Flow Control is designed to maintain balanced message processing, preventing the overloading of any participating parties.
- Receive Maximum: Max number of unacknowledged PUBLISH messages the client can accommodate
- "Receive Maximum" value is not specified, the default value of 65,535
- Helps with not overloading low resourced IoT

Resource: https://www.hivemq.com/blog/mqtt5-essentials-part12-flow-control/

Enhanced Authentication



- Challenge-response
  - vs credential-based
- Authentication methods:
  - Salted Challenge Response Authentication Mechanism (SCRAM)
  - Kerberos
- Enabled throughout the entire message flow
- Uses new AUTH packets

Resource: https://www.hivemq.com/blog/mqtt5-essentials-part11-enhanced-authentication/

MQTT: Brokers

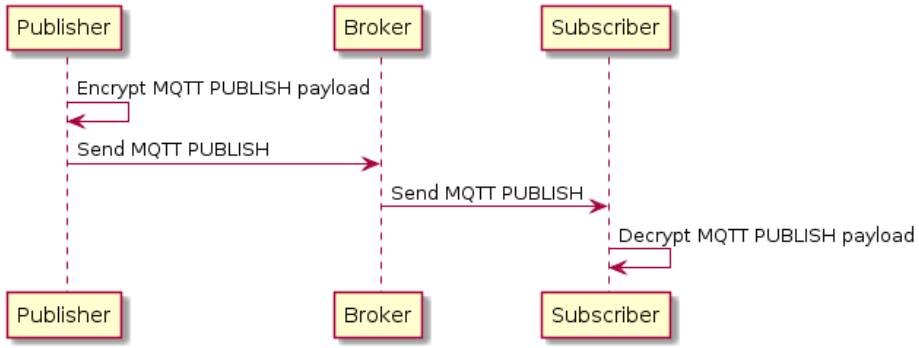
SINGAPORE INSTITUTE OF TECHNOLOGY

Appliance	Cloud	Open Source
IBM MessageSight 1m connections 15m QoS 0 / sec policies for security, messaging, connection developer VM	**HiveMQ** **IBM IoT Foundation** **Eurotech EDC** **Litmus Loop** Others	**Mosquitto** (C) **Mosca** (Node.js) **Moquette** (Java) **RSMB** (C) [tiny] Others **Eclipse Sandbox** iot.eclipse.org
Commercial	"Freemium"	Free

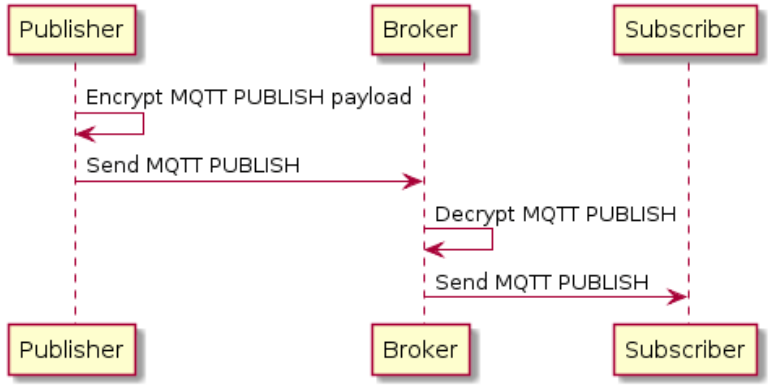
Outside MQTT - Security

deprecated

- **Transport Layer Security** (TLS) ~~and **Secure Sockets Layer** (SSL):~~
  - Encrypts the network connection between client ↔ broker.
  - Prevents eavesdropping, MITM attacks, and tampering.
  - Ensures the broker you're connecting to is genuine (certificate verification).
  - Works ***before*** any MQTT message is processed.
- **Payload Encryption** is the encryption of application-specific data on the application level
  - End-to-end (E2E) encryption



  - Client-to-broker



Outside MQTT - (Access Control List) ACL

- **ACL** = Access Control List implemented by the broker, ***not part of MQTT***.
- **Defines** who can publish and who can subscribe to which topics.
- **Enforced** inside the broker as part of its authorization system.
- **Prevents** unauthorized reading/writing of MQTT data streams.

- **Security**: Stop malicious or unintended clients from publishing harmful data.
- **Privacy**: Ensure only authorized clients can subscribe to sensitive topics.
- **Multi-tenancy**: Separate customers, departments, or device groups with topic-level permissions.
- **Stability**: Prevent accidental flooding (e.g., a device publishing to #).
- **Integrity**: Enforce strict rules such as "device can only publish to its own topic."

User: sensorA

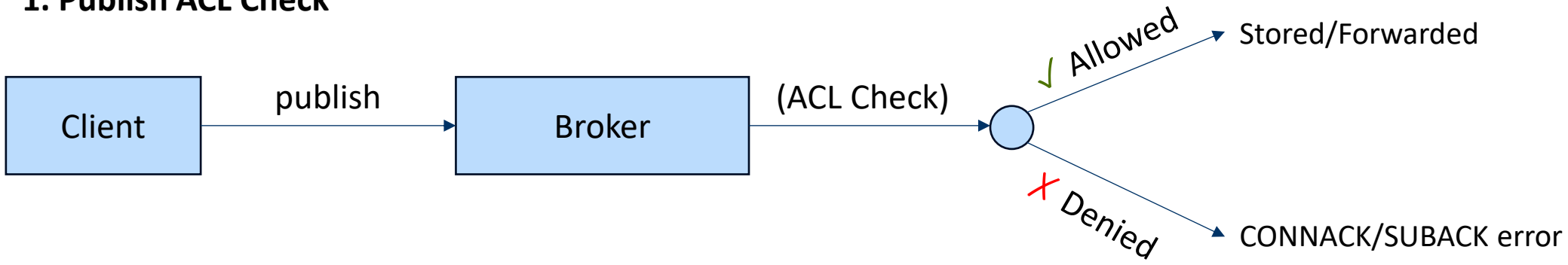
- Allow:
  - sensorA/data (publish)
  - sensorA/status (publish)
  - control/sensorA/# (subscribe)
- Deny:
  - sensorB/#
  - factory/control/#
  - # (wildcards not allowed unless explicitly permitted)

User: dashboardApp

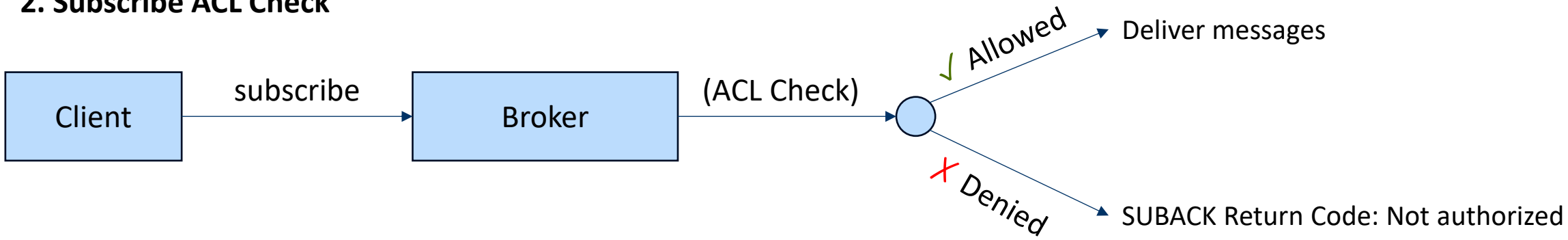
- Allow:
  - +/data (subscribe)
  - factory/alerts/# (subscribe)
- Deny:
  - Any publish operations

How a Broker Enforces ACLs

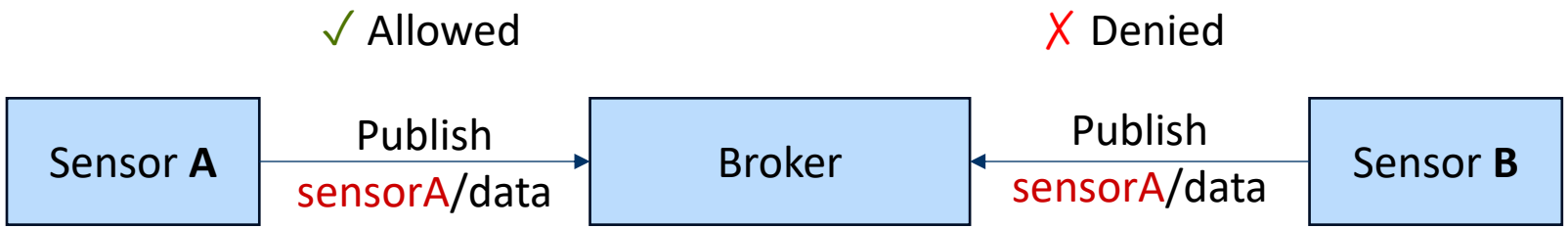
1. Publish ACL Check



2. Subscribe ACL Check



ACL Protecting Topic Access

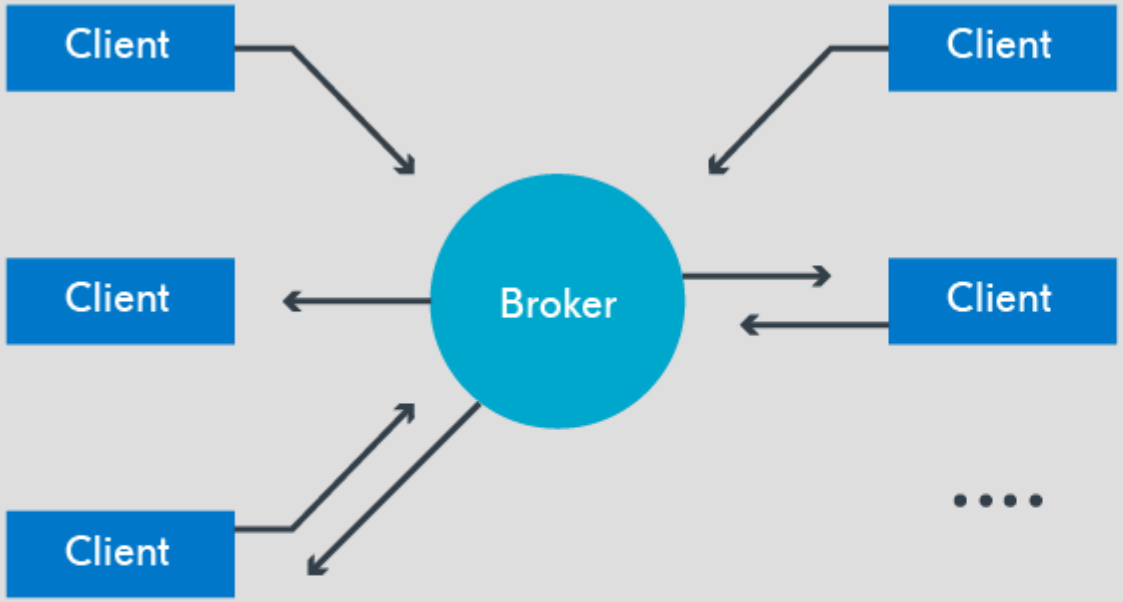


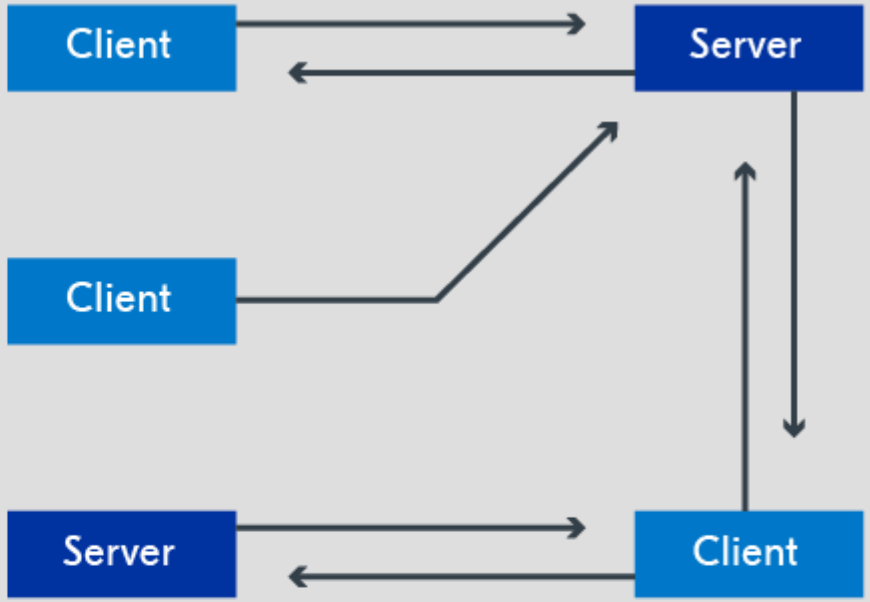
Summary - MQTT

- **Lightweight Protocol** for IoT designed for constrained environments.
- **Publish/Subscribe model**: Decouples publishers from subscribers, simplifying communication without requiring direct connections.
- **Three QoS Levels**
- **Flexible and efficient**: Ideal for unreliable or bandwidth-limited networks
- **Security via TLS/~~SSL~~**: Secure communication channels to prevent unauthorised access.
  - SSL = old, insecure, deprecated
  - TLS = new, secure, the real protocol used today
- **Widely adopted** from hobby projects to large-scale IoT solutions.

MQTT vs CoAP

RFC 7252





MQTT > CoAP

1. **Simplicity and Ease of Implementation:** MQTT is often perceived as simpler to implement compared to CoAP
2. **Reliability in Unstable Networks**: MQTT QoS levels allow for message delivery guarantees, which is crucial in many IoT applications.
3. **Better Support for High-Latency Networks**: MQTT works well over networks with high latency.
4. **Widespread Adoption and Ecosystem**: MQTT has a broader adoption in the industry.
5. **Scalability**: MQTT's publish-subscribe model scales well for a large number of devices.
6. **Interoperability with Existing Infrastructure**: MQTT’s use of TCP/IP makes it easy to integrate.
7. **Use Cases and Industry Preferences**: Certain industries have preferred MQTT for its reliability and ease of integration, especially in scenarios like telemetry, where data needs to be pushed from a large number of devices to a server.
8. **Strong Community and Standards Support**: MQTT has strong support from organisations like OASIS and the Eclipse Foundation, and it's an ISO standard (ISO/IEC 20922), which adds to its credibility and reliability.

https://www.hivemq.com/blog/mqtt-vs-coap-for-iot/

MQTT < CoAP

1. **Extremely Low-Power Devices**: CoAP is designed for very low-power devices that need to conserve battery life aggressively. It works well in scenarios where devices need to remain in sleep mode for extended periods and wake up only to send or receive brief messages.
2. **Devices with Limited Processing Capabilities**: CoAP is more straightforward and requires less processing power compared to MQTT, which is an advantage in environments with very limited computational resources.
3. **UDP-Based Networks**: CoAP operates over UDP, which is generally lighter on resources than TCP (used by MQTT). In environments where TCP's overhead is undesirable or unsustainable, CoAP is the better choice.
4. **Direct Web Integration**: CoAP's RESTful design and built-in proxying support to HTTP make it inherently suitable for direct integration with web technologies. This is beneficial in constrained environments where devices need to interact directly with web-based services without the additional overhead of a MQTT broker.
5. **Multicast Support**: CoAP supports multicast requests natively, allowing a single message to be sent to multiple devices simultaneously. This is particularly useful in scenarios like firmware updates or mass configuration changes in a constrained network.
6. **Low-Latency Device-to-Device Communication**: In scenarios where devices need to communicate directly with each other with minimal latency, CoAP's lightweight, stateless operation can be advantageous compared to MQTT, which generally requires communication through a broker.

つづく